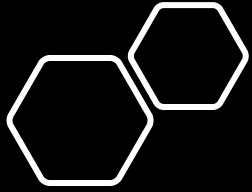


Machine learning

Support vector machine (SVM)

Kernel SVM

Images by Dhilip Subramanian



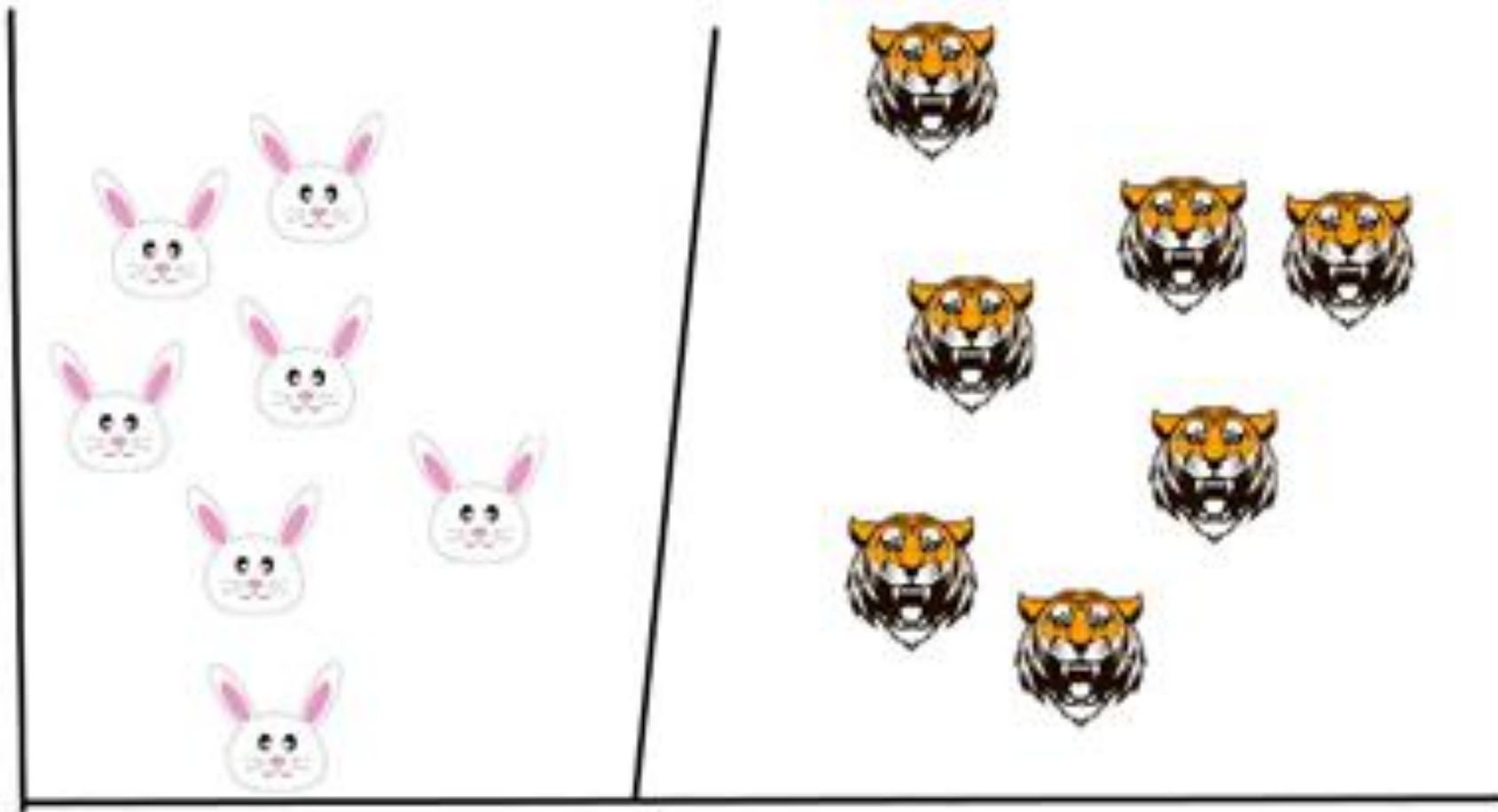
Support vector machine (SVM)

- Support Vector Machine is one of the most popular machine learning algorithms
- SVMs were extremely popular around the time they were developed (in the 1990s)
- SVM continue to be the go-to method for a high performing algorithm with little tuning

How to separate the animals?



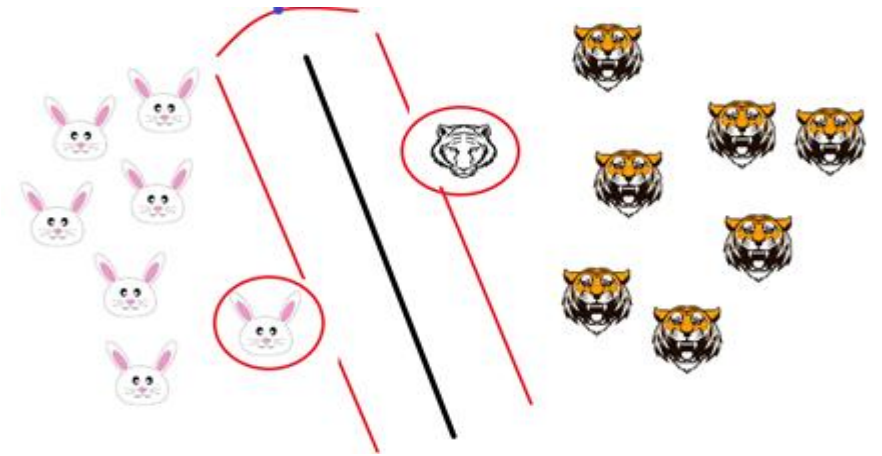
How to separate the animals?



What is SVM?

- Support vector machines, also called SVM, is a ***supervised learning algorithm***
- Can be used for classification and regression problems
 - Support vector classification (SVC)
 - Support vector regression (SVR)
- It is **used for small datasets** as it takes too long to process

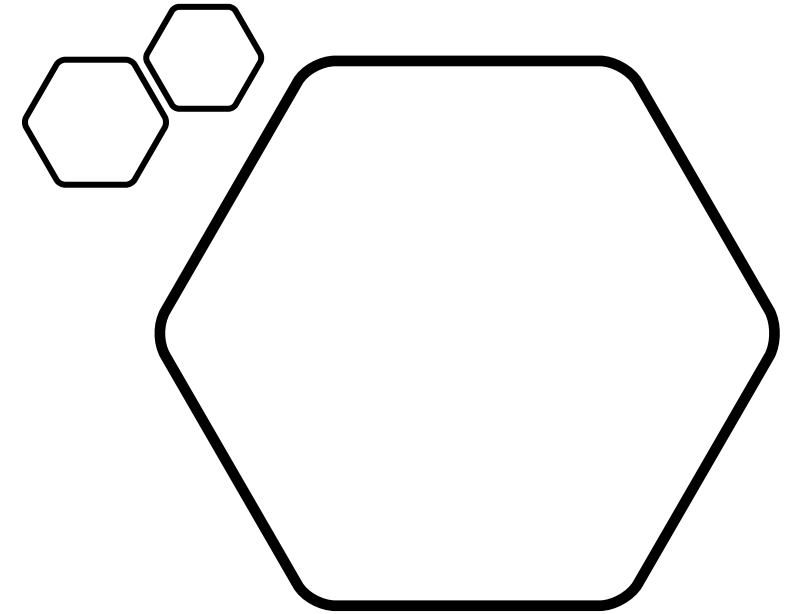
SVM 101



- Imagine you have a bunch of dots on a piece of paper, and you want to draw a line that separates these dots into different groups. SVM is a mathematical technique that helps us find the best line to do that.
- In SVM, we call the dots "data points" and we have two groups: positive and negative. The line we draw is called a "hyperplane." The goal of SVM is to find the hyperplane that maximizes the distance between the closest data points from both groups. We want this line to have the biggest "margin" possible.

Binary Classification Problem

- Input space: $X \subseteq R^n$
- Output space: $y = \{1, -1\}$
- Training data: $S = \{(X_1, y_1), \dots, (X_m, y_m)\}$
- S is drawn i.i.d from some distribution D
- **Goal:** Select a hypothesis $h \in H$ that best predicts other points drawn i.i.d from D



Binary classification

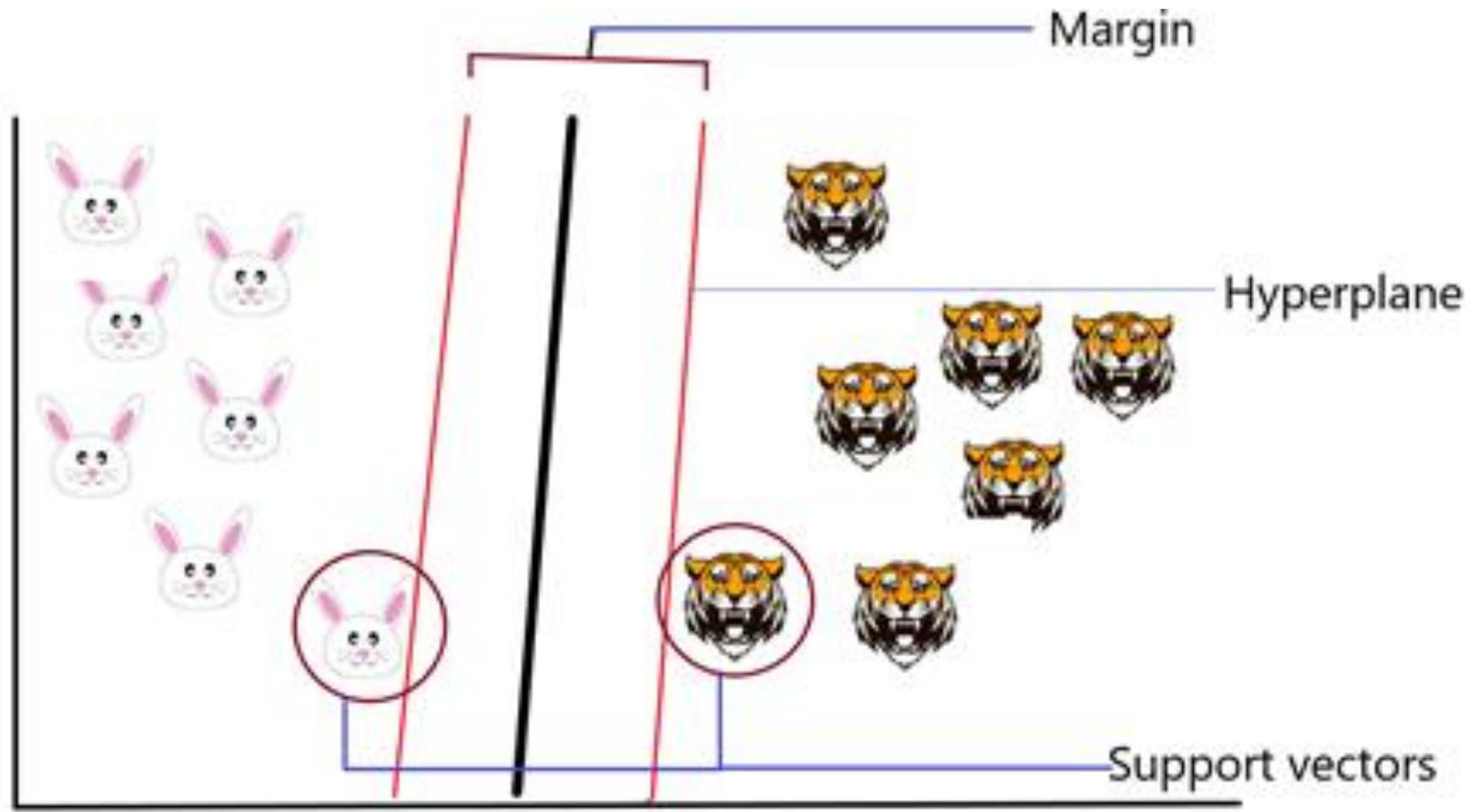
- Infinitely many ways to classify
- Occam's razor – simple classification rules provide better results (e.g., linear classifier or hyperplane)

- Our class of linear classifiers:

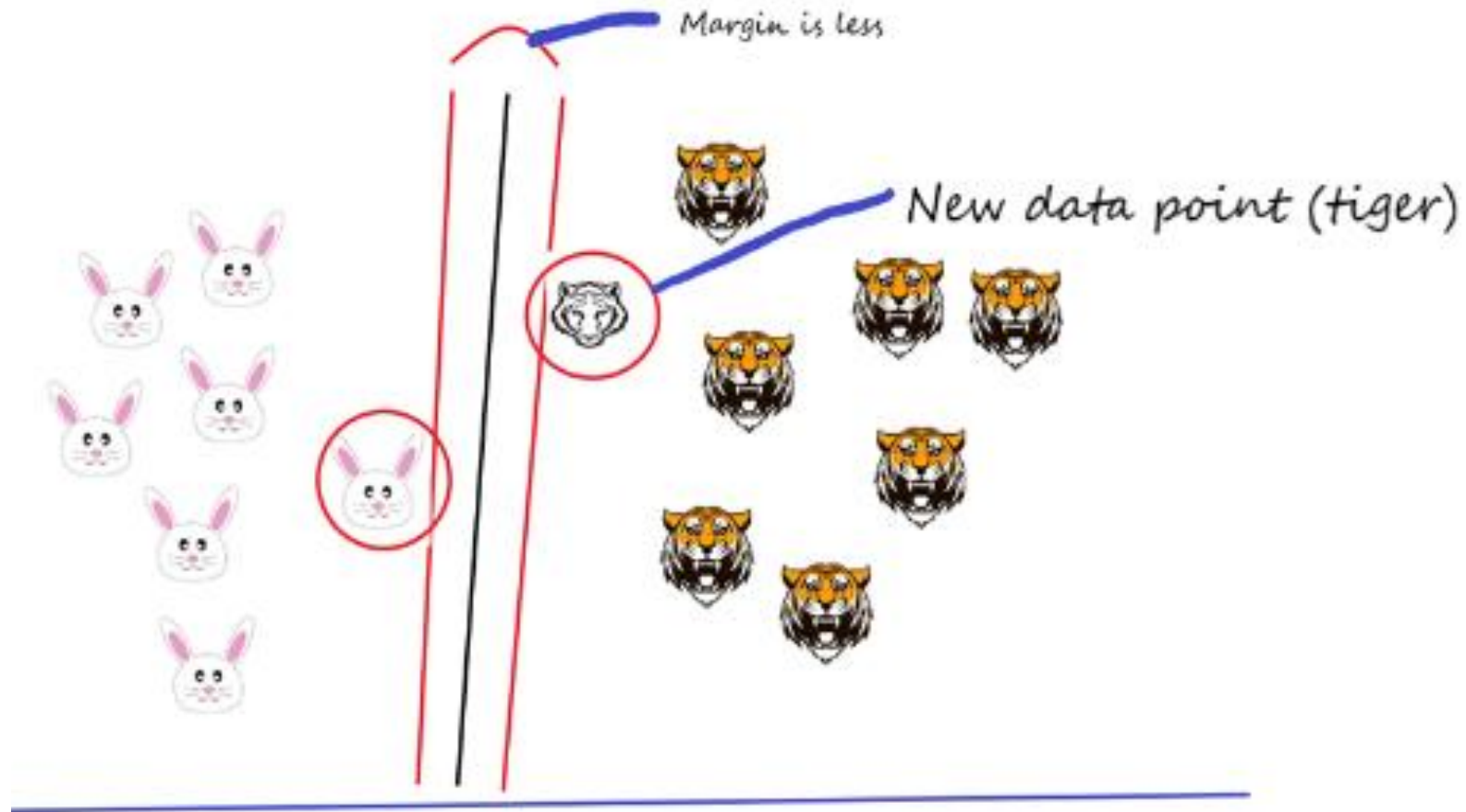
$h \in H$ maps $x \in X$ to $+1$ if $(w * x + b) \geq 0$

$$H = \{x \rightarrow \text{sign}(w * x + b) | w \in R^n, b \in R\}$$

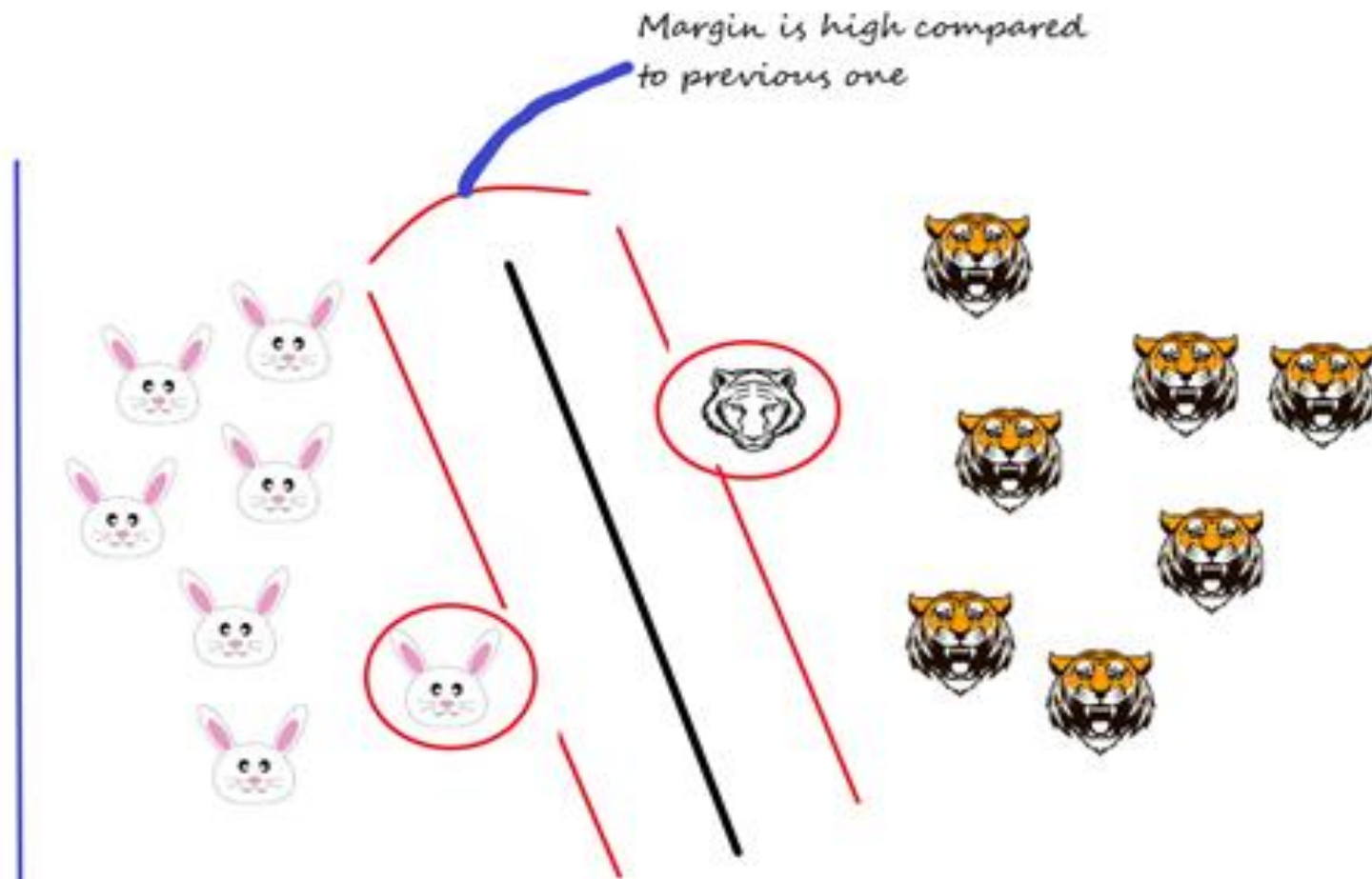
Maximum margin



Adding new data point



Adding new data point



BRACE YOURSELF...

MATH IS COMING...

makeameme.org

Step 1: Representing the data

- We can represent each data point with a vector.
- Let's say we have two features for each point, like the x and y coordinates.
- So, a point (2, 3) can be represented as a vector [2, 3].

Step 2: Finding the hyperplane

- Now, we want to find the equation of a line that separates the positive and negative data points.

This line equation can be written as: $w_1 * x + w_2 * y + b = 0$

- Here, w_1 and w_2 are weights that determine the direction of the line, and b is a bias that moves the line up or down.

Step 3: Maximizing the margin

- To find the best line, we want to maximize the distance between the closest points from both groups to the hyperplane. These points are called "**support vectors**"
- We calculate the distance between the hyperplane and a point using a formula:

$$distance = (w_1 * x + w_2 * y + b) / \sqrt{w_1^2 + w_2^2}$$

Step 4: Optimization

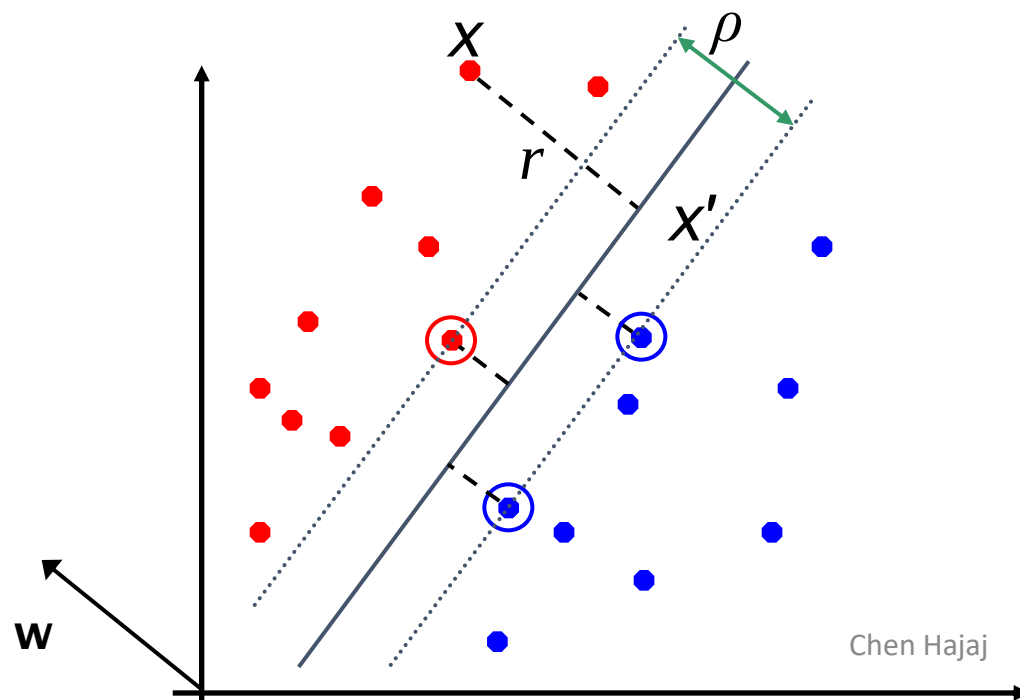
- SVM aims to find the best values for w_1 , w_2 , and b .
- We can think of this as an optimization problem, where we try to find the maximum margin while keeping the misclassification errors as low as possible.
- This optimization process involves minimizing a mathematical function called a "cost function" that penalizes misclassifications.
- The SVM algorithm uses various mathematical techniques to find the best values for w_1 , w_2 , and b .

Step 5: Making predictions

- Once we have found the optimal hyperplane, we can use it to make predictions for new, unseen data points.
- We simply plug the values of the new point into the line equation $(w_1 * x + w_2 * y + b)$ and check which side of the line it falls on.
That tells us the predicted group.

Geometric Margin

- Distance from the example to the separator is $r = y(\mathbf{w}^T \mathbf{x} + b)/|\mathbf{w}|$
- Examples closest to the hyperplane are the support vectors
- Margin ρ of the separator is the width of separation between support vectors of classes



Example

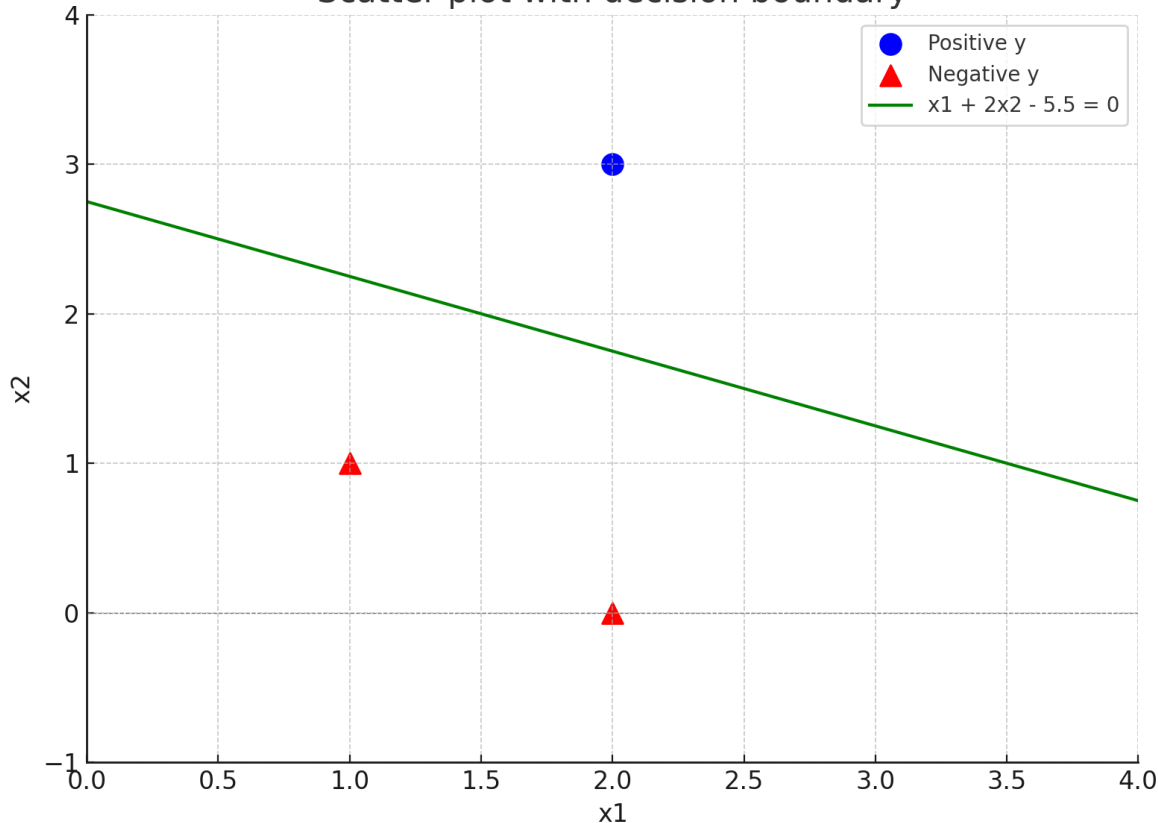
Geometric solution: decision boundary will be parallel to the shortest line connecting points of the two classes giving a weight vector of (1,2)

The optimal decision surface is orthogonal to that line and intersects it at the halfway point. Therefore, it passes through (1.5,2)

$$x_1 + 2x_2 + b = 0 \rightarrow 1.5 + 2 * 2 + b = 0 \rightarrow b = -5.5$$

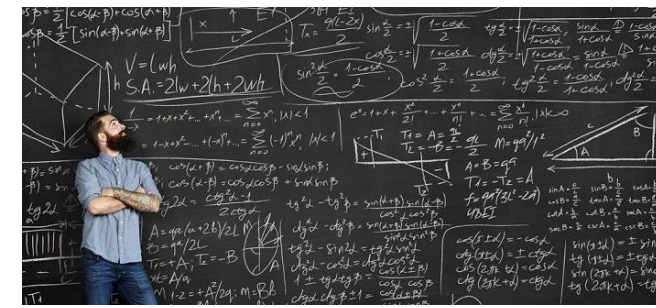
The SVM boundary is: $y = x_1 + 2x_2 - 5.5$

Scatter plot with decision boundary



x_1	x_2	y	$x_1 + 2x_2 - 5.5$
1	1	-2.5	-2.5
2	0	-3.5	-3.5
2	3	2.5	2.5

Linear SVMs Mathematically (cont.)



- Then we can formulate the quadratic optimization problem:

Find $\mathbf{w}$ and b such that

$$r = \frac{2}{\|\mathbf{w}\|} \text{ is maximized; and for all } \{(\mathbf{x}_i, y_i)\}$$
$$\mathbf{w}^T \mathbf{x}_i + b \geq 1 \text{ if } y_i = 1; \quad \mathbf{w}^T \mathbf{x}_i + b \leq -1 \text{ if } y_i = -1$$

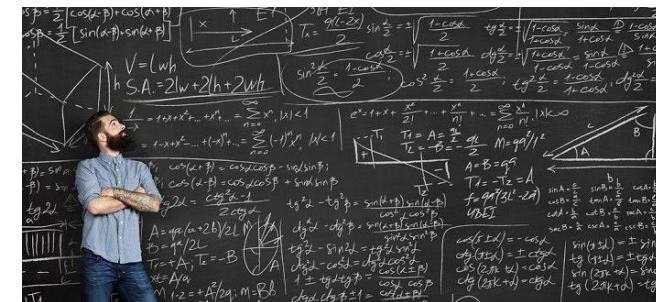
- A better formulation ($\min ||\mathbf{w}|| = \max 1/ ||\mathbf{w}||$):

Find $\mathbf{w}$ and b such that

$\Phi(\mathbf{w}) = \frac{1}{2} \mathbf{w}^T \mathbf{w}$ is minimized;

and for all $\{(\mathbf{x}_i, y_i)\}$: $y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1$

Solving the Optimization Problem



Find $\mathbf{w}$ and b such that
 $\Phi(\mathbf{w}) = \frac{1}{2} \mathbf{w}^T \mathbf{w}$ is minimized;
and for all $\{(\mathbf{x}_i, y_i)\}$: $y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1$

- This is now optimizing a quadratic function subject to linear constraints
- Quadratic optimization problems are a well-known class of mathematical programming problem, and many (intricate) algorithms exist for solving them (with many special ones built for SVMs)
- The solution involves constructing a dual problem where a Lagrange multiplier α_i is associated with every constraint in the primary problem:

Find $\alpha_1 \dots \alpha_N$ such that
 $\mathbf{Q}(\boldsymbol{\alpha}) = \sum \alpha_i - \frac{1}{2} \sum \sum \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$ is maximized and
(1) $\sum \alpha_i y_i = 0$
(2) $\alpha_i \geq 0$ for all α_i

Error Analysis Using Leave-One-Out

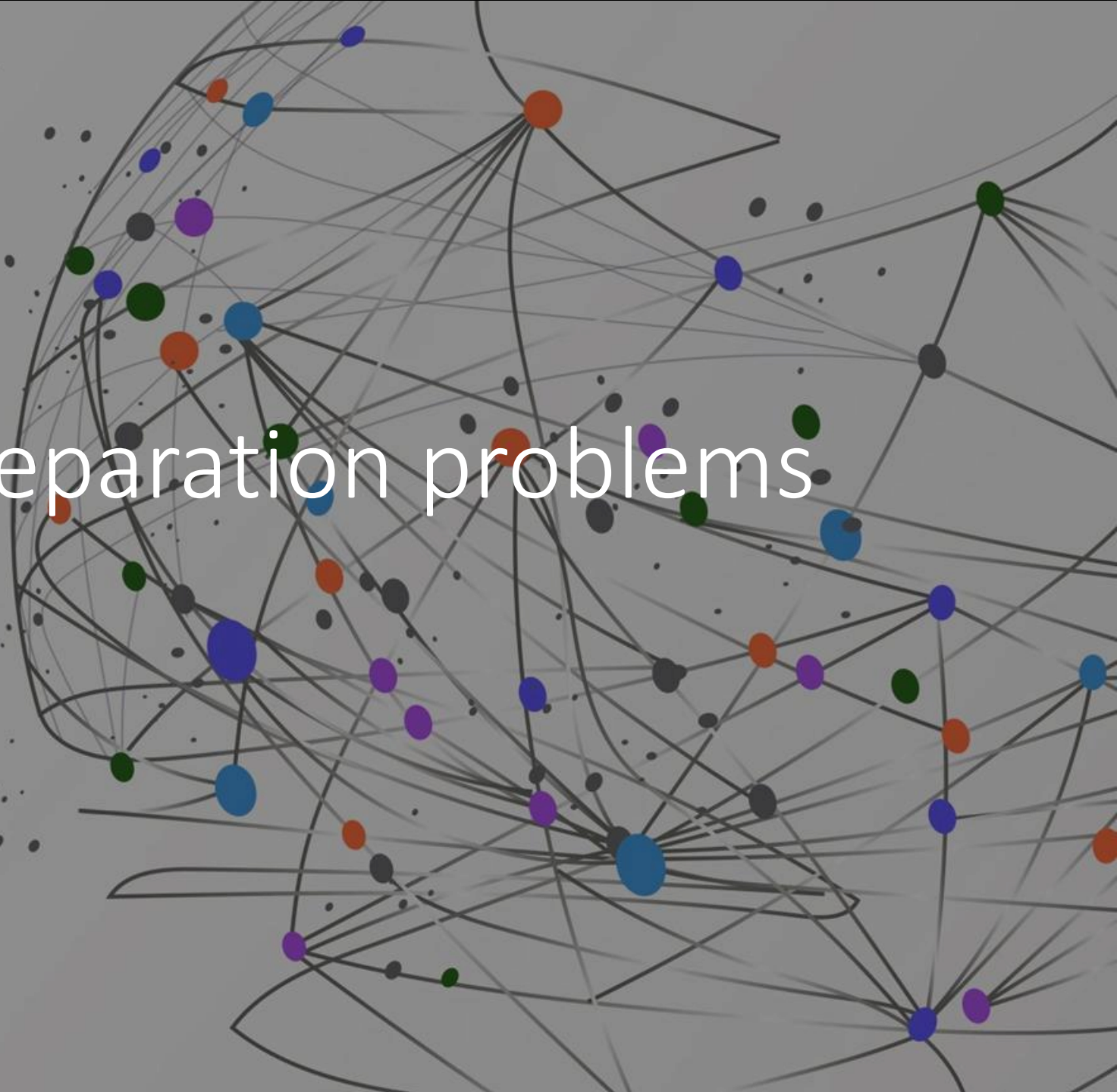
The Leave-One-Out (LOO) method

- Remove one point at a time from the training set
- Train an SVM for the remaining points
- Test our model using the removed point

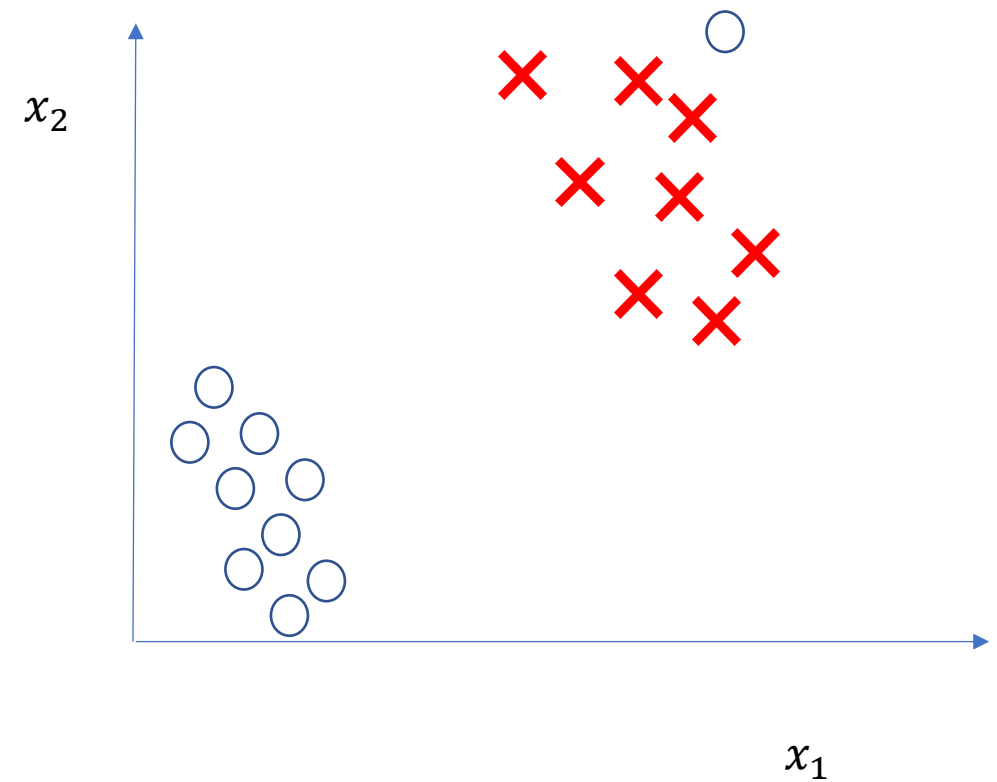
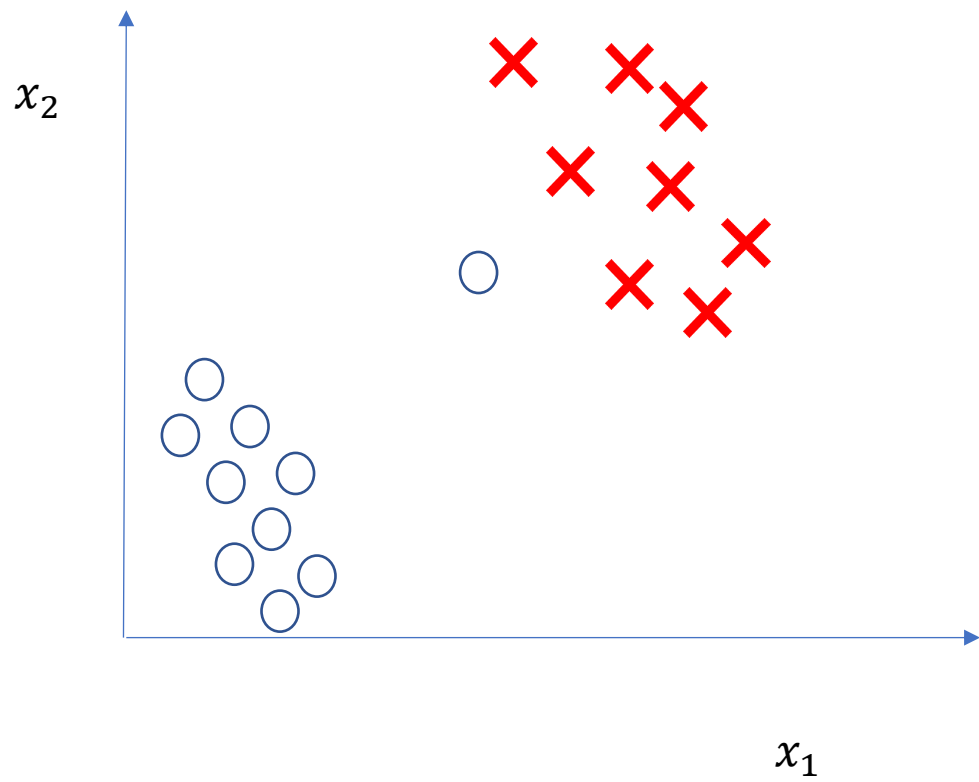
Definition

- $\hat{R}_{LOO} = \frac{1}{m} \sum_{i=1}^m I(h_{S-\{x^i\}}(x^i) \neq y^i)$
- The indicator function $I(\text{exp})$ is 1 if exp is true, otherwise 0

Non-linear separation problems

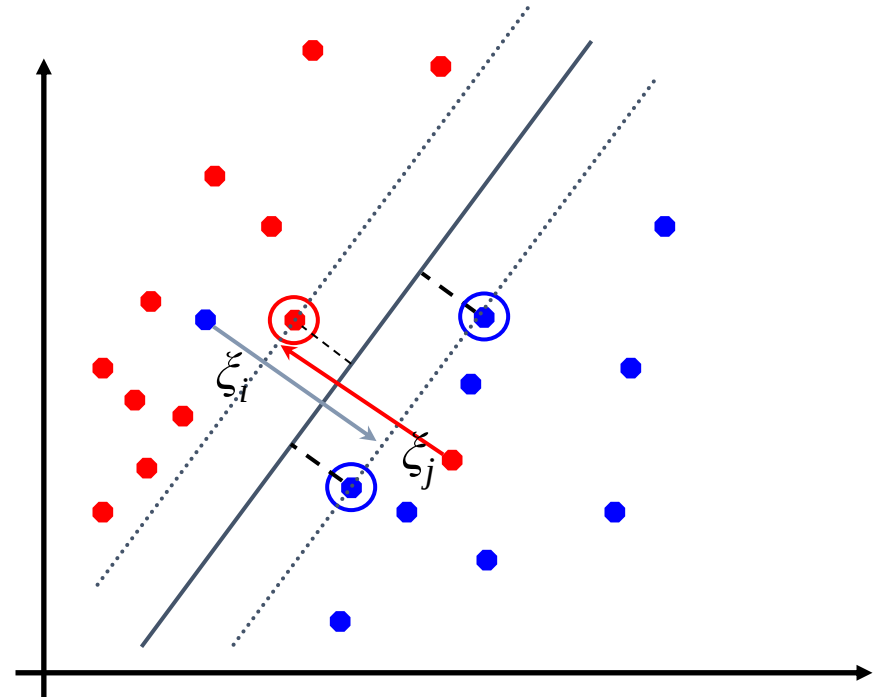


What will you do?



Soft Margin Classification

- If the training data is not linearly separable, slack variables ξ_i can be added to allow misclassification of difficult or noisy examples.
- **Allow some errors**
 - Let some points be moved to where they belong, at a cost
- Still, try to minimize training set errors, and to place hyperplane “far” from each class (large margin)



Soft Margin Classification Mathematically

- **The old formulation:**

Find $\mathbf{w}$ and b such that

$\Phi(\mathbf{w}) = \frac{1}{2} \mathbf{w}^T \mathbf{w}$ is minimized and for all $\{(\mathbf{x}_i, y_i)\}$

$y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1$

- **The new formulation incorporating slack variables:**

Find $\mathbf{w}$ and b such that

$\Phi(\mathbf{w}) = \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum \xi_i$ is minimized and for all $\{(\mathbf{x}_i, y_i)\}$

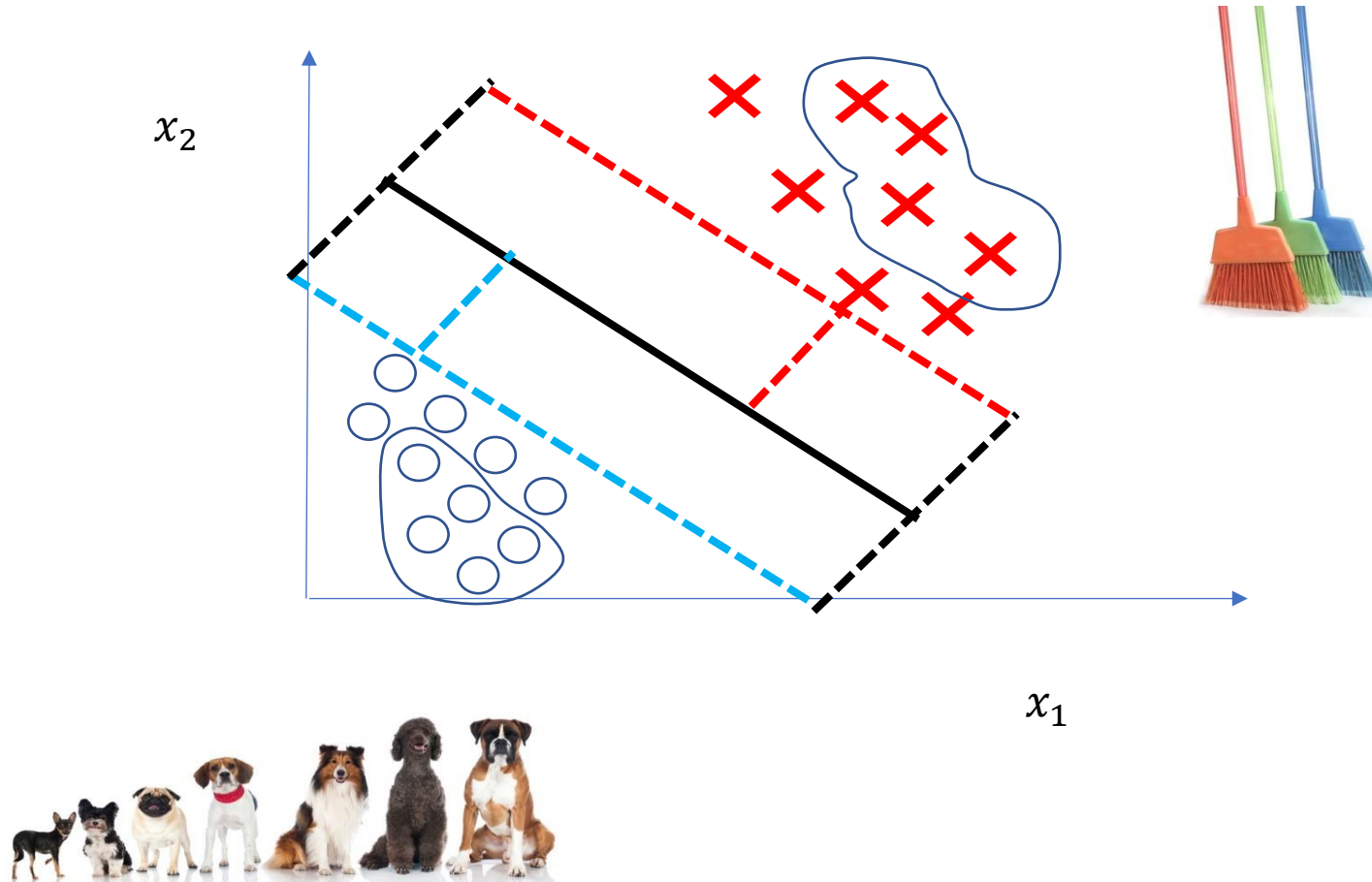
$y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i$ and $\xi_i \geq 0$ for all i

- Parameter C can be viewed as a way to control overfitting (a regularization term)

What's so special about SVM?

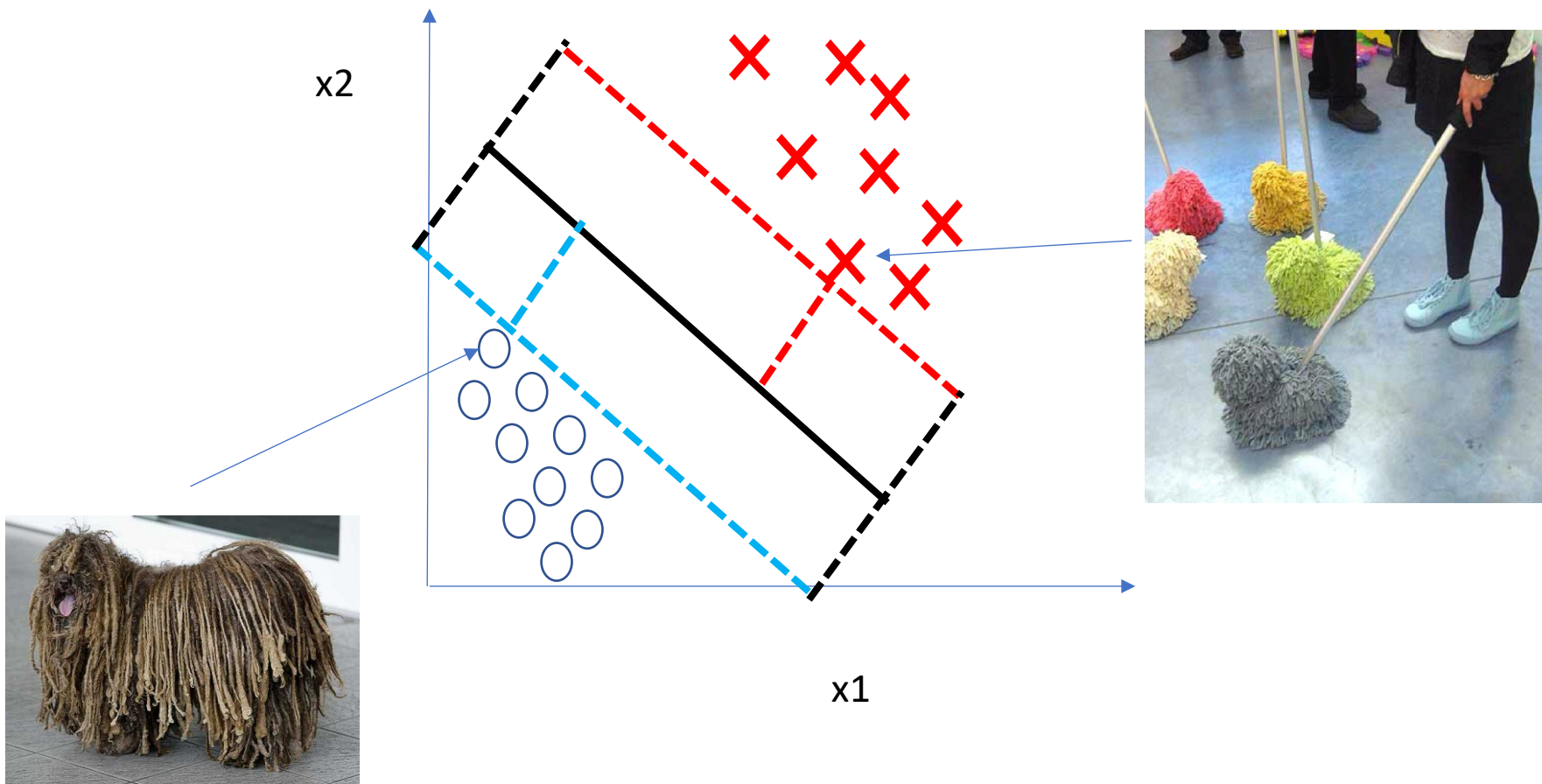


What's so special about SVM?



**Traditional
classification
algorithms**

What's so special about SVM?



Puppy or bagel



Chihuahua or muffin



Sheepdog or mop



Labradoodle or fried chicken



Kernel SVM

How to separate the points?



- when data is linearly separable - SVM

NOT linearly separable



*how do I
separate this
data?*



NOT linearly separable data

HIGHER DIMENSIONAL SPACE

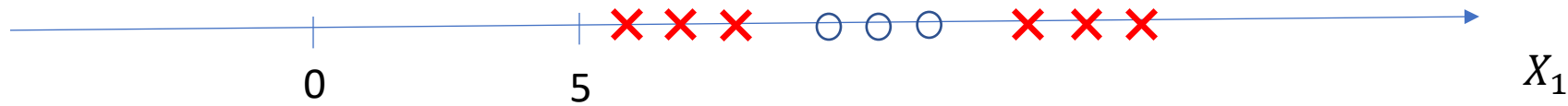


THE KERNEL TRICK



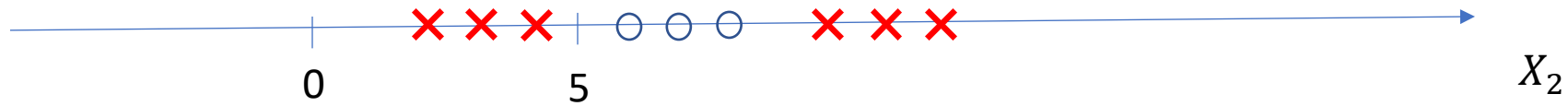
Mapping to a HIGHER DIMENSIONAL SPACE

The data is NOT linearly separable

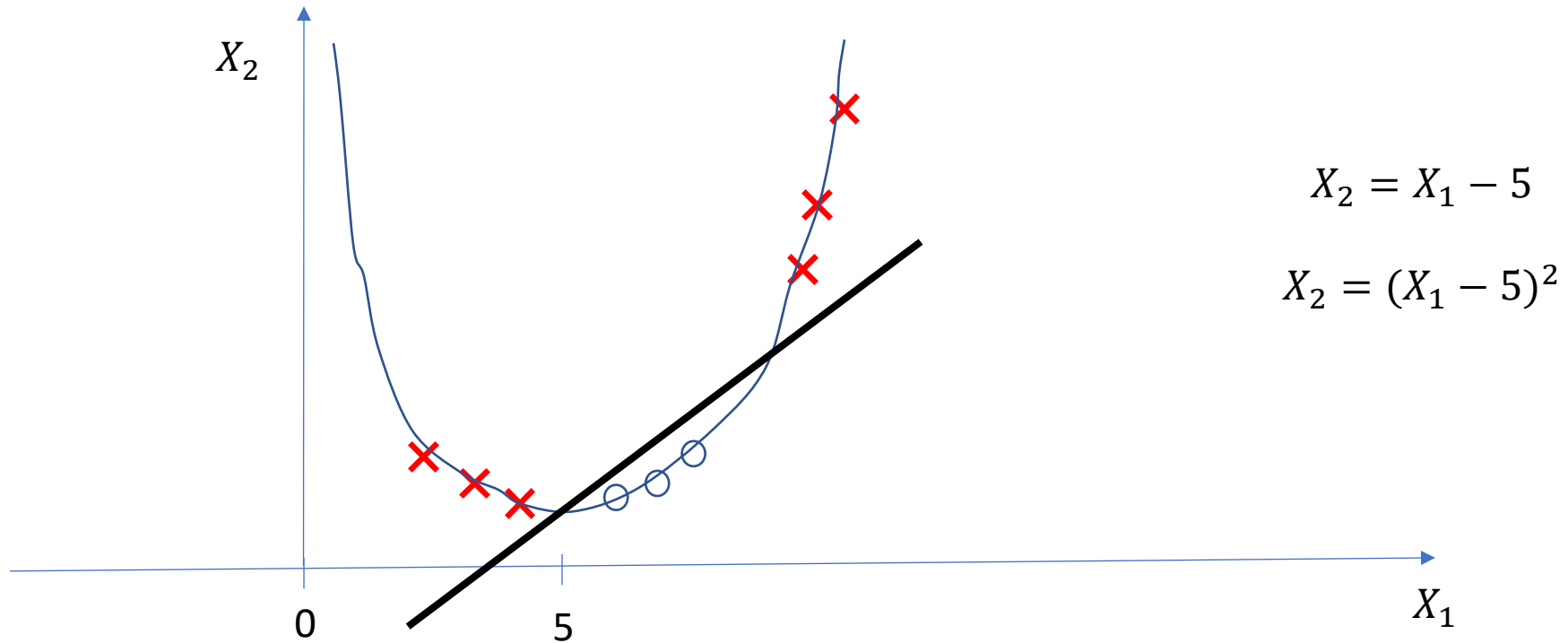


Mapping to a HIGHER DIMENSIONAL SPACE

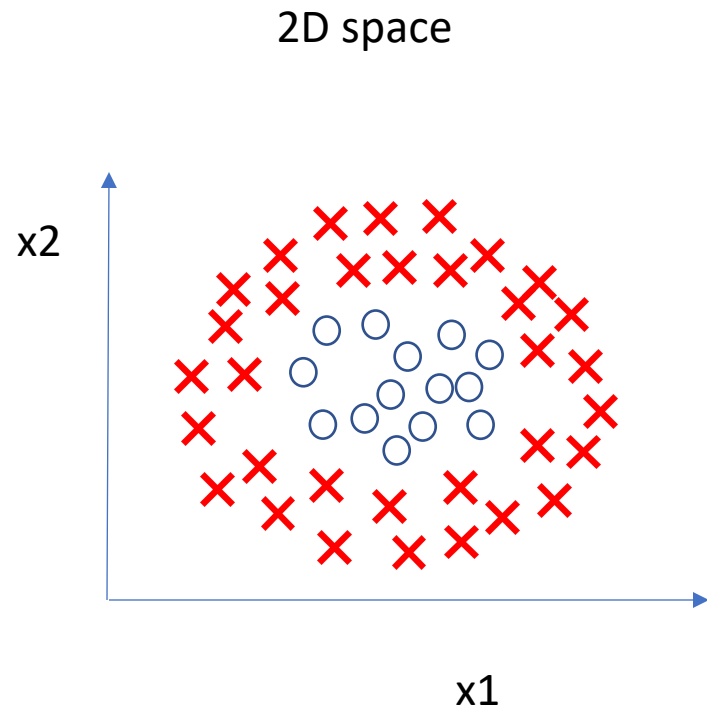
$$X_2 = X_1 - 5$$



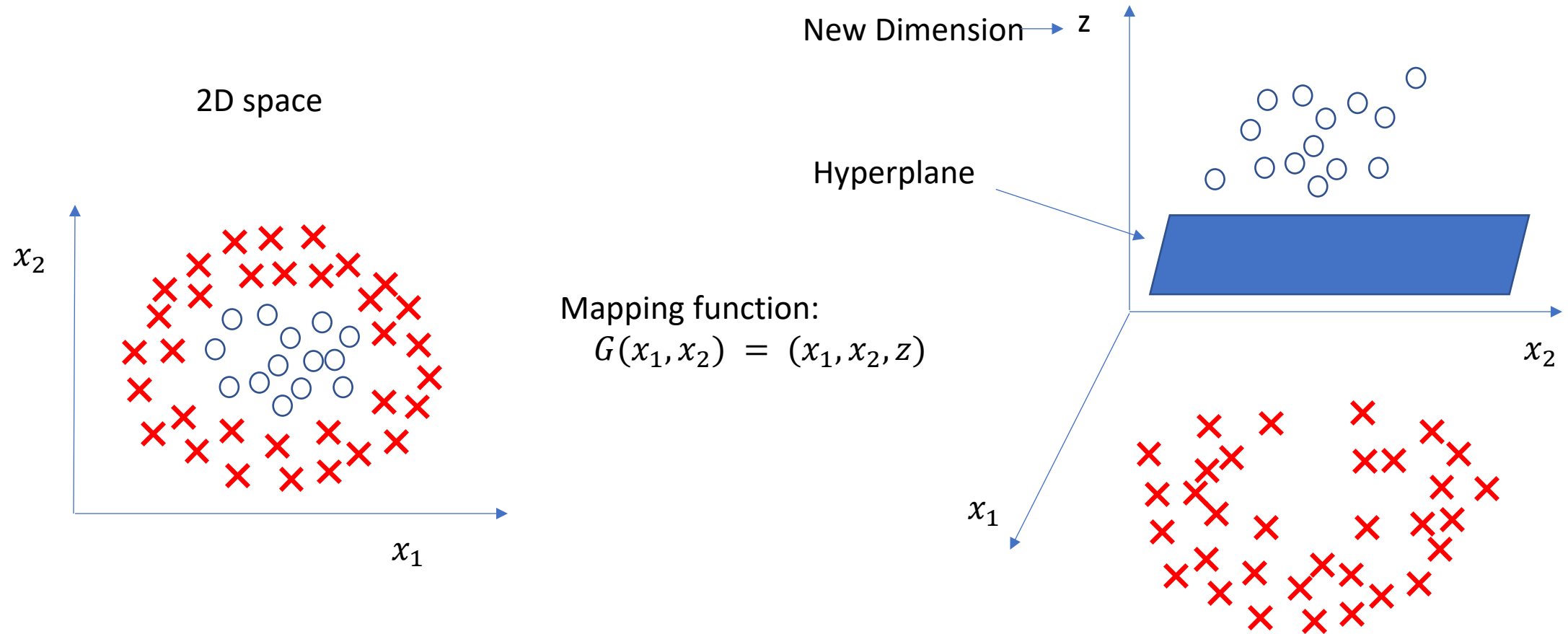
Mapping to a HIGHER DIMENSIONAL SPACE



Mapping to a HIGHER DIMENSIONAL SPACE



Mapping to a HIGHER DIMENSIONAL SPACE

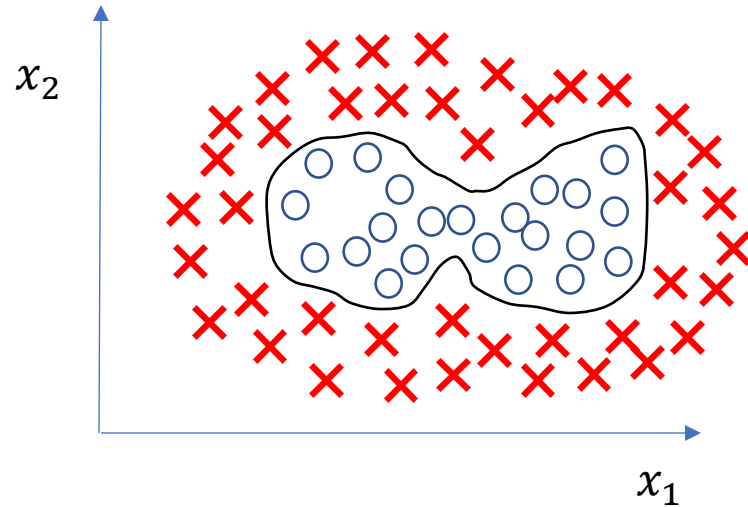




But there is a catch

Mapping to higher dimensional space can be computationally expensive

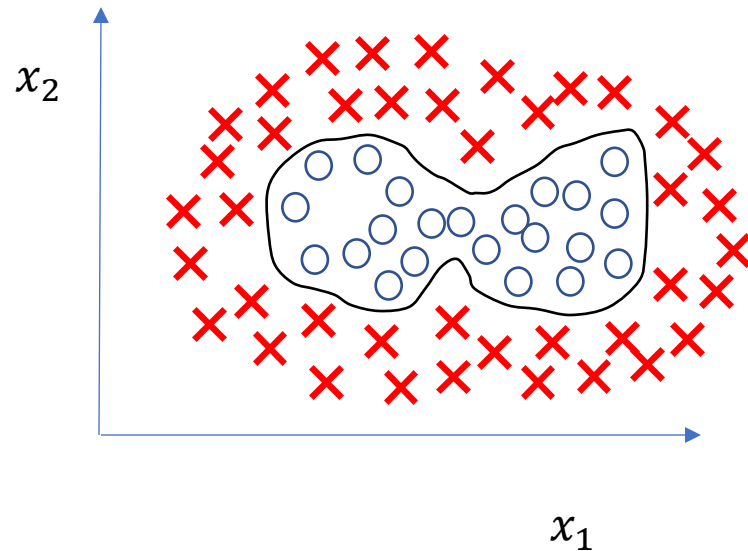
Kernels intuition



Predict **×** if:

$$\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_1 x_2 + \beta_4 x_1^2 + \beta_5 x_2^2 + \dots \geq 0$$

Kernels intuition



Predict **×** if:

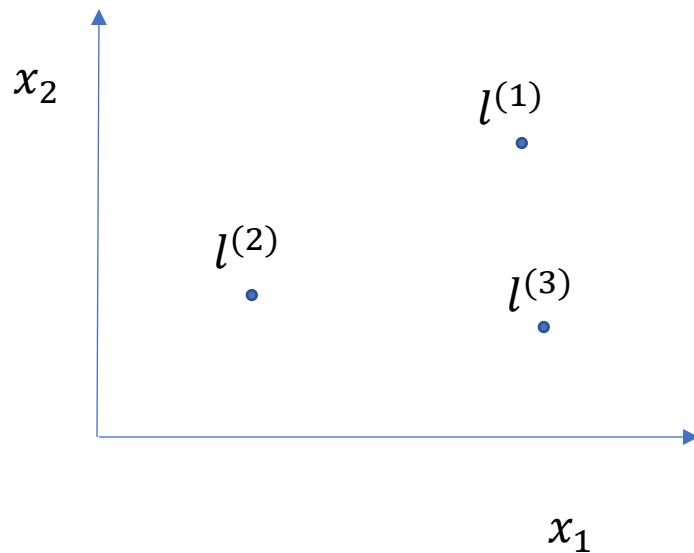
$$\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_1 x_2 + \beta_4 x_1^2 + \beta_5 x_2^2 + \dots \geq 0$$



$$\beta_0 + \beta_1 f_1 + \beta_2 f_2 + \beta_3 f_3 + \beta_4 f_4 + \beta_5 f_5 + \dots \geq 0$$

Is there a better choice of the features $f_1, f_2, f_3 \dots$?
Maybe non-polynomial features?

Kernel function



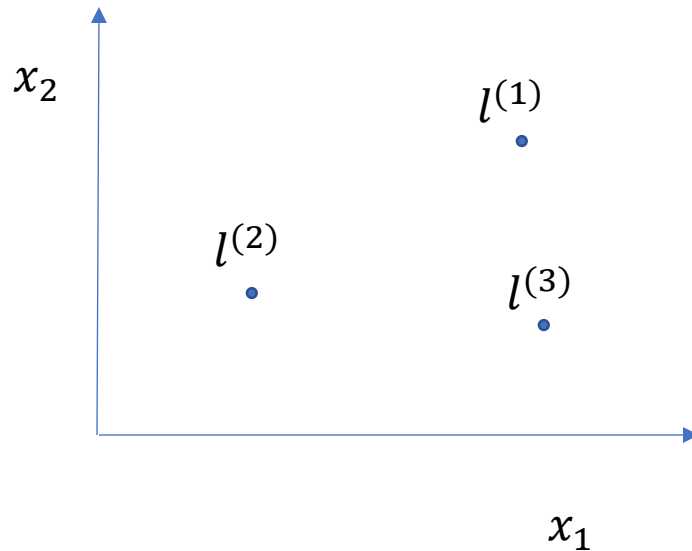
We will define the features ($f_1, f_2, f_3..$) differently

First step: pick data points on the graph – **landmarks**
 $l^{(1)}, l^{(2)}, l^{(3)}$

Kernel “trick”

- The idea behind the kernel trick is to use a kernel function that operates on pairs of data points and calculates the similarity or dissimilarity between them in the higher-dimensional space.
- The kernel function takes the original data points as inputs and returns a measure of their similarity in the transformed space.

Kernel function

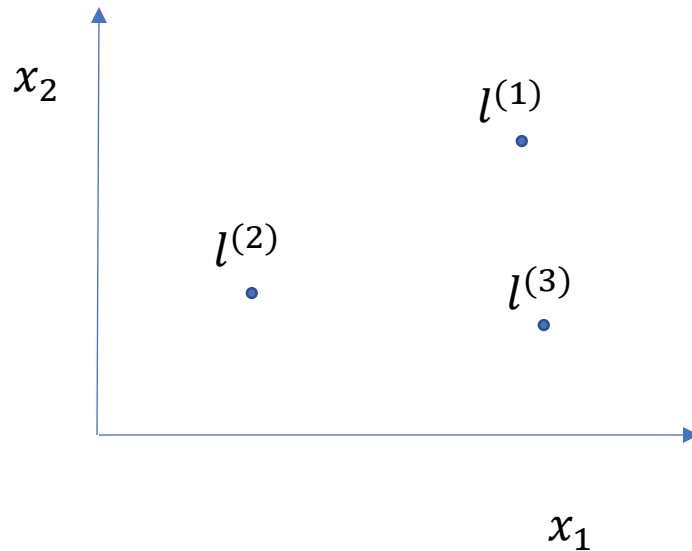


We will define the features ($f_1, f_2, f_3..$) differently.

First step: pick data points on the graph – **landmarks**
 $l^{(1)}, l^{(2)}, l^{(3)}$

$$\text{similarity}(x, l^{(i)}) = K(x, l^{(i)})$$
$$e^{-\frac{\|x - l^{(1)}\|^2}{2\sigma^2}} = \text{Gaussian kernel}$$

Kernel function



We will define the features ($f_1, f_2, f_3..$) differently

First step: pick data points on the graph – **landmarks**
 $l^{(1)}, l^{(2)}, l^{(3)}$

$$\text{Given } x: f_1 = \text{similarity}(x, l^{(1)}) = e^{-\frac{\|x - l^{(1)}\|^2}{2\sigma^2}}$$

$$f_2 = \text{similarity}(x, l^{(2)}) = e^{-\frac{\|x - l^{(2)}\|^2}{2\sigma^2}}$$

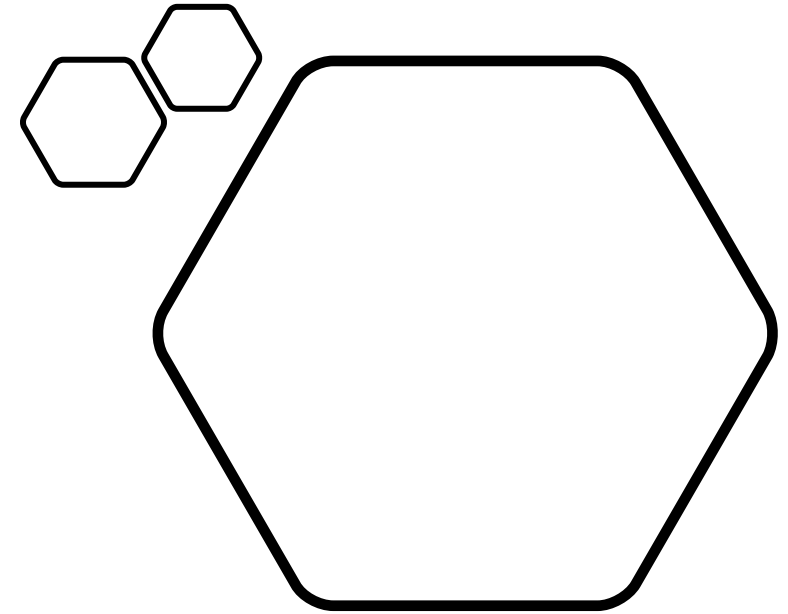
$$f_3 = \text{similarity}(x, l^{(3)}) = e^{-\frac{\|x - l^{(3)}\|^2}{2\sigma^2}}$$

Better understanding Gaussian kernel

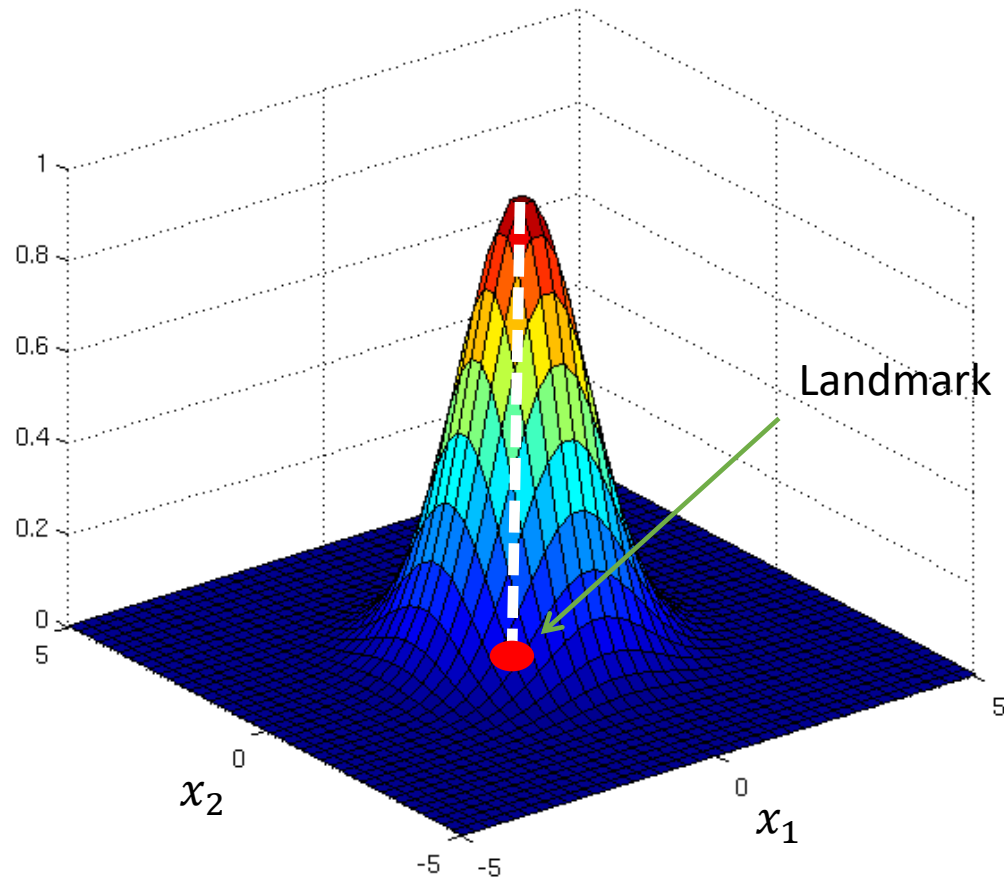
$$\text{similarity}(x, l^{(i)}) = e^{-\frac{\|x - l^{(i)}\|^2}{2\sigma^2}}$$

If x is near $l^{(i)}$: $\text{similarity}(x, l^{(i)}) = e^{-\frac{0}{2\sigma^2}} \approx 1$

If x is far from $l^{(i)}$: $\text{similarity}(x, l^{(i)}) = e^{-\frac{\text{Large number}}{2\sigma^2}} \approx 0$

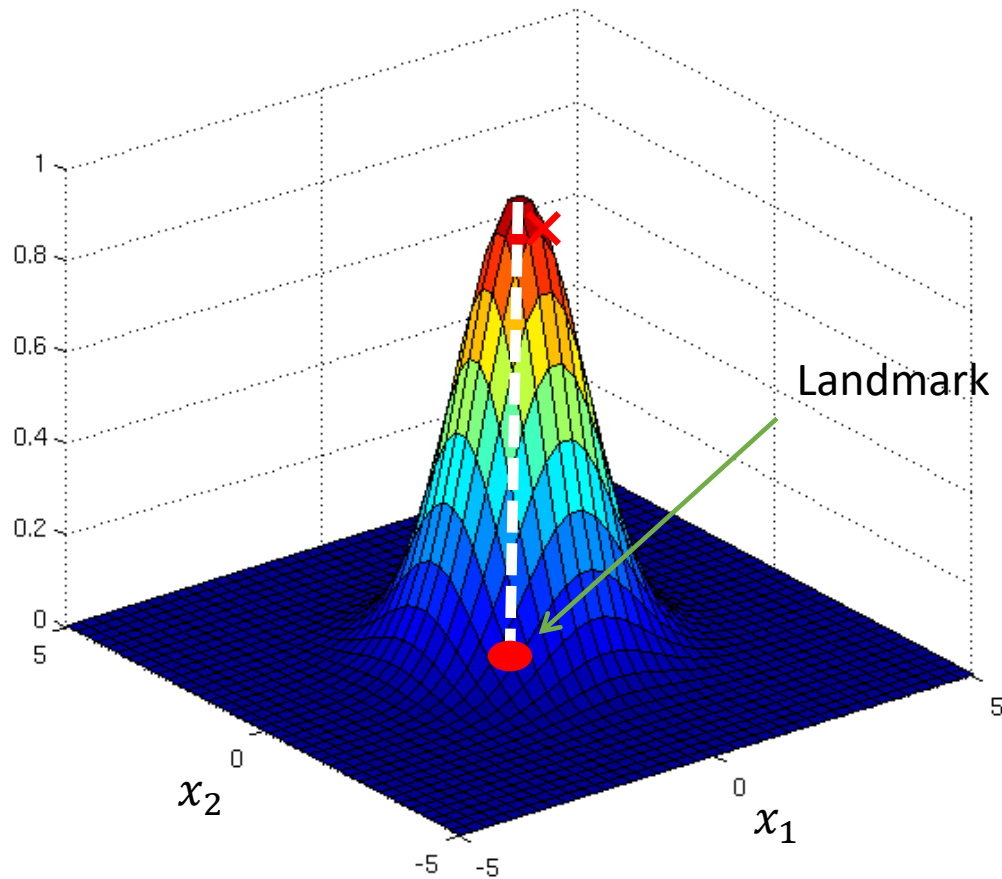


The Gaussian RBF kernel



If we pick the landmark to be $[0,0]$, and σ to be 1 what will be the similarity value to the point $[0.1,0.1]$?

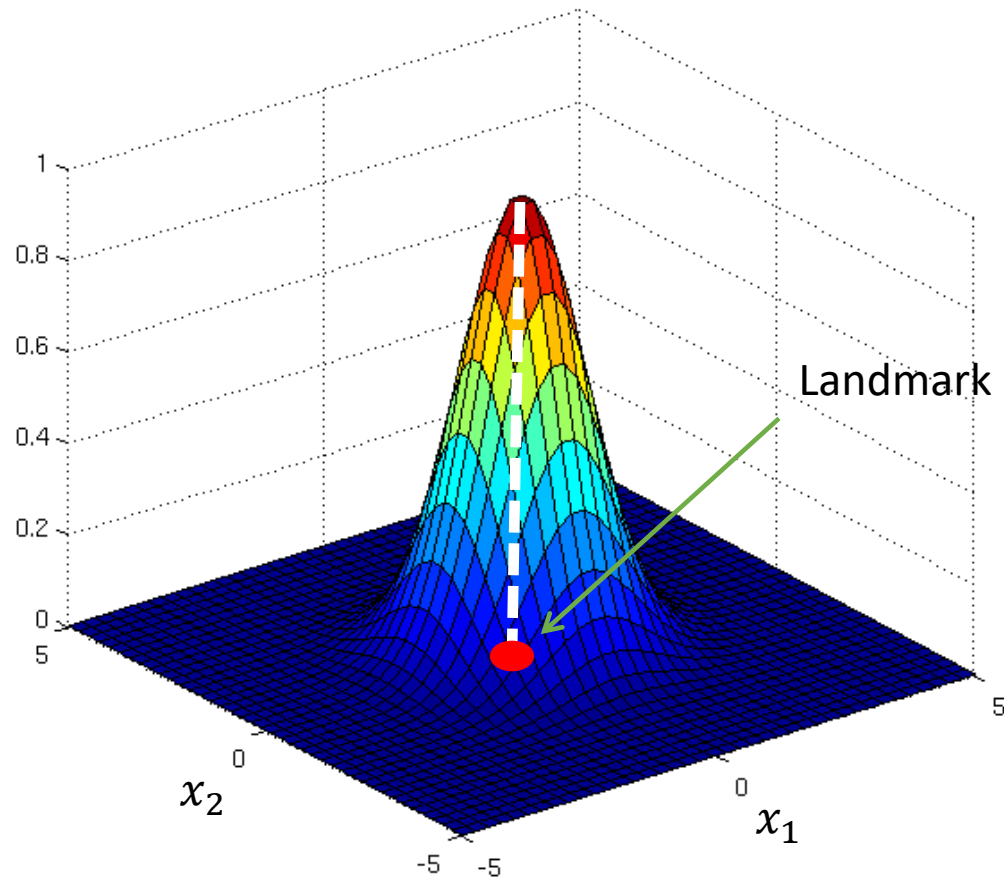
The Gaussian RBF kernel



If we pick the landmark to be $[0,0]$, and σ to be 1 what will be the similarity value to the point $[0.1,0.1]$?

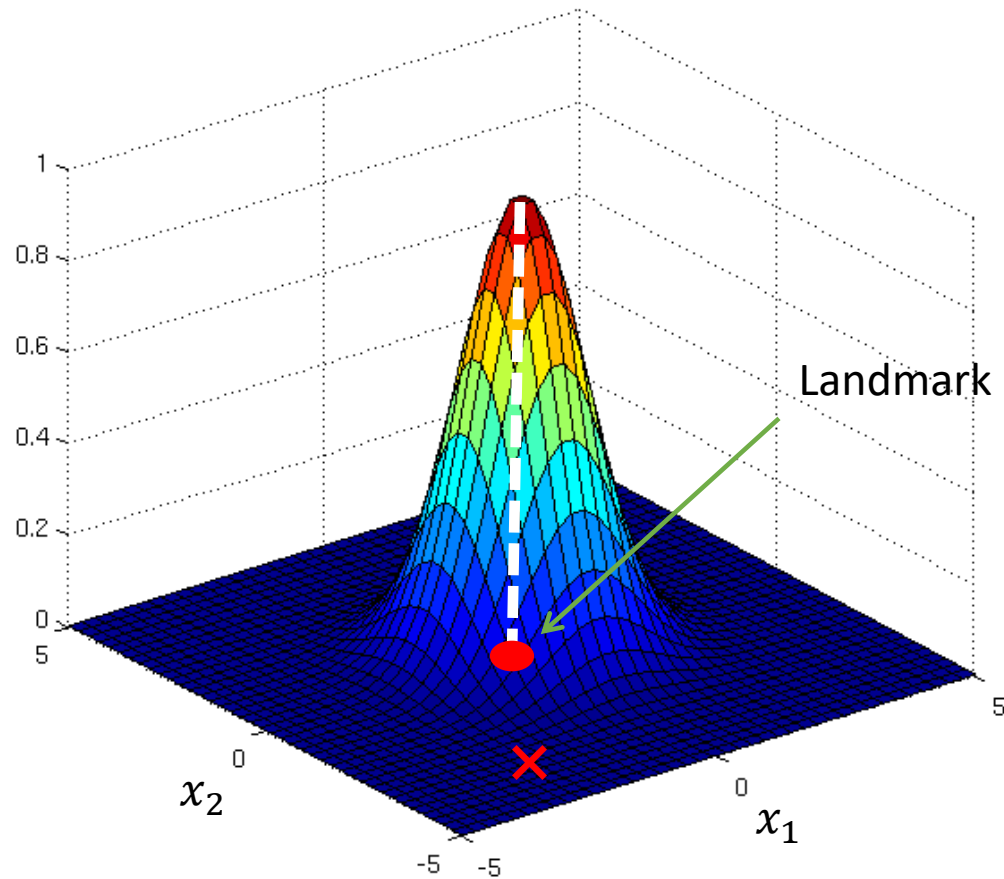
$$\begin{aligned} K([0.1,0.1],[0,0]) &= e^{-\frac{\|x-l^{(1)}\|^2}{2\sigma^2}} = e^{-\frac{\sqrt{0.1^2+0.1^2}^2}{2}} = e^{-\frac{0.14}{2}} \\ &= e^{-0.07} = 0.93 \end{aligned}$$

The Gaussian RBF kernel



If we pick the landmark to be $[0,0]$, and σ to be 1 what will be the similarity value to the point $[-3,-4]$?

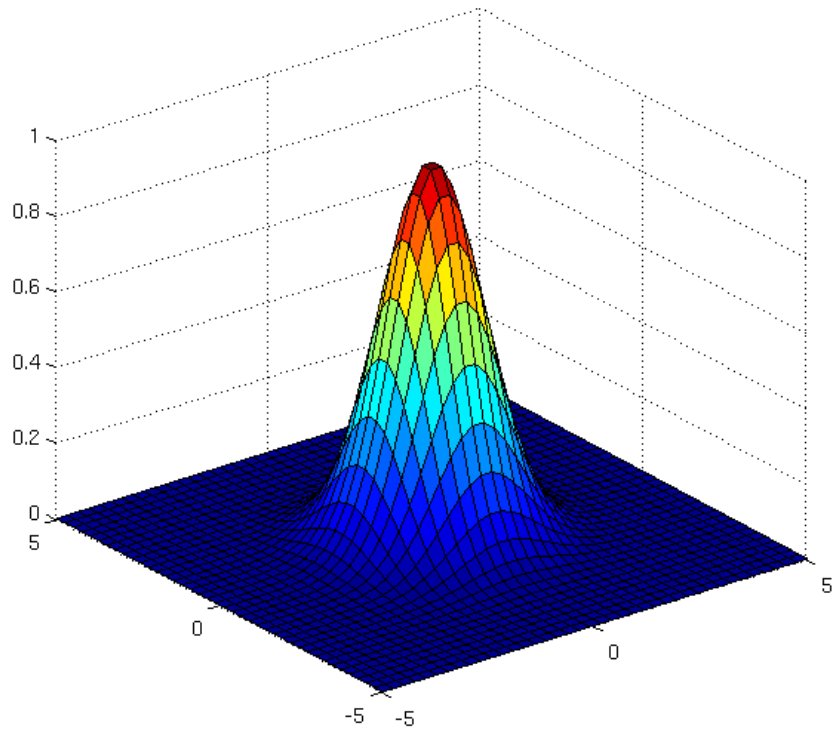
The Gaussian RBF kernel



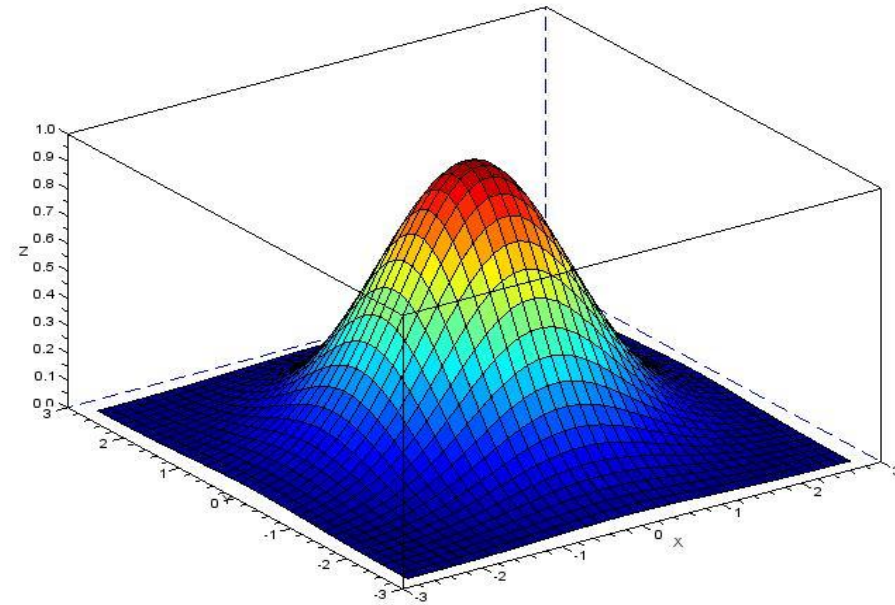
If we pick the landmark to be $[0,0]$,
and σ to be 1 what will be the
similarity value to the point $[-3,-4]$?

$$K([-3,-4],[0,0]) = e^{-\frac{\|x-l^{(1)}\|^2}{2\sigma^2}} = e^{-\frac{\sqrt{-3^2 + -4^2}}{2}} = e^{-\frac{5}{2}}$$
$$= e^{-2.5} = 0.082$$

What is the role of σ ?

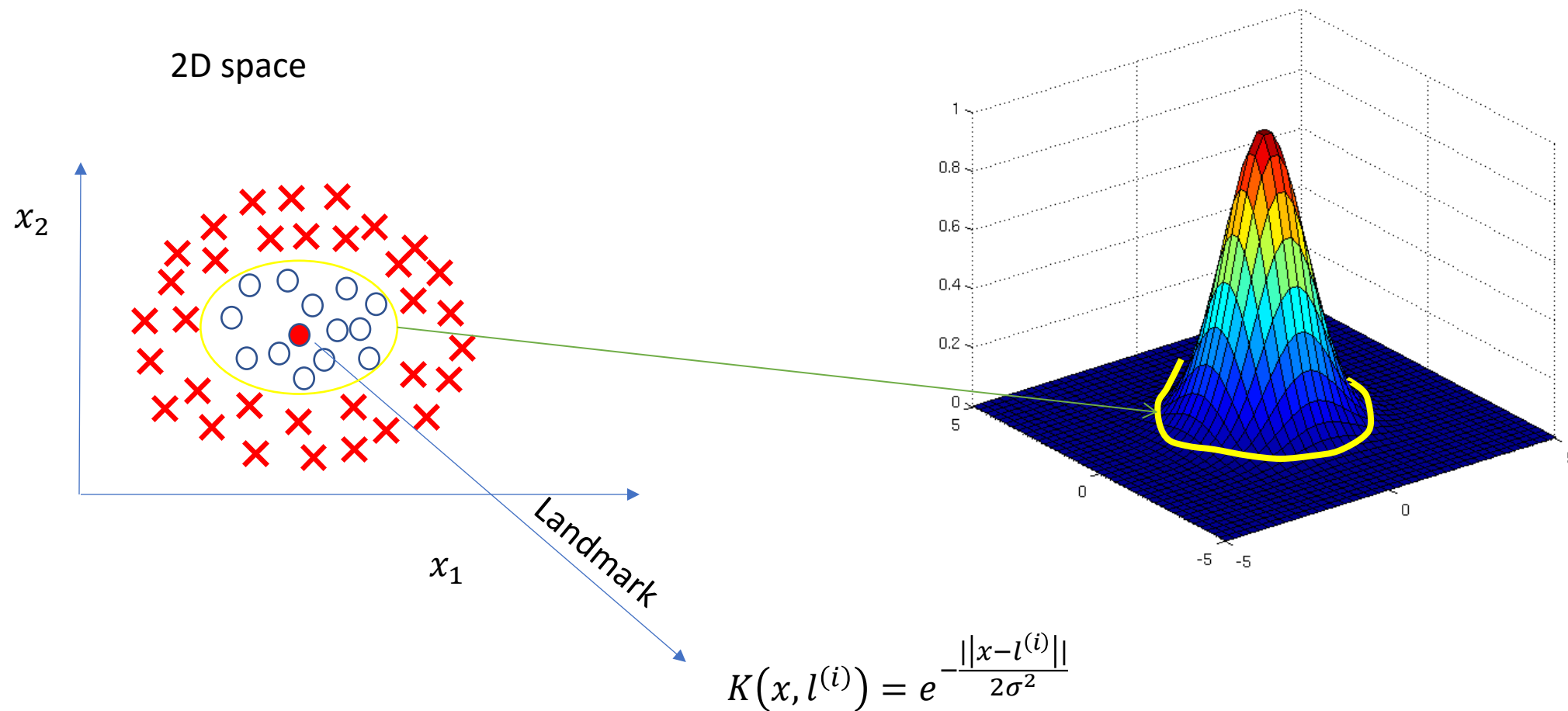


$\sigma = 1$

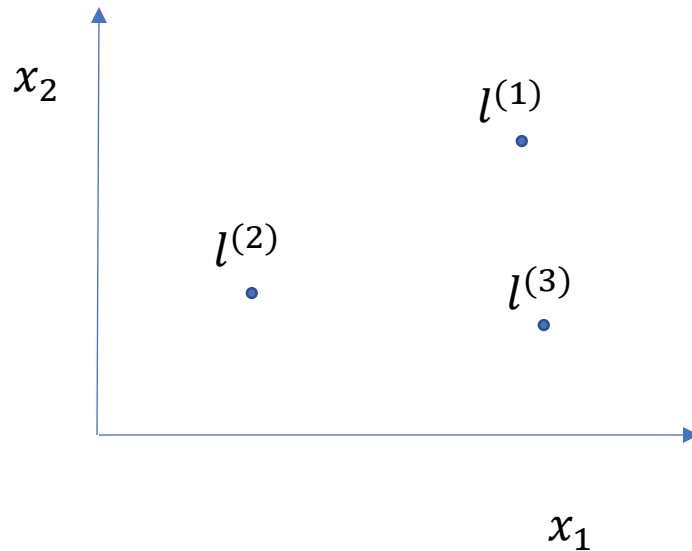


$\sigma = 3$

Landmark intuition



Kernel function

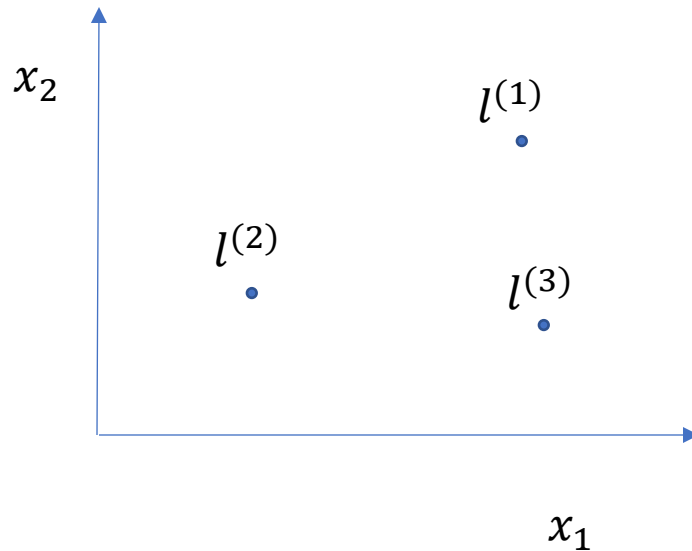


We will define the features ($f_1, f_2, f_3..$) differently

First step: pick data points on the graph – **landmarks**
 $l^{(1)}, l^{(2)}, l^{(3)}$

$$\begin{aligned}\text{Given } x: f_1 &= \text{similarity}(x, l^{(1)}) = e^{-\frac{\|x - l^{(1)}\|^2}{2\sigma^2}} \\ f_2 &= \text{similarity}(x, l^{(2)}) = e^{-\frac{\|x - l^{(2)}\|^2}{2\sigma^2}} \\ f_3 &= \text{similarity}(x, l^{(3)}) = e^{-\frac{\|x - l^{(3)}\|^2}{2\sigma^2}}\end{aligned}$$

Kernel function

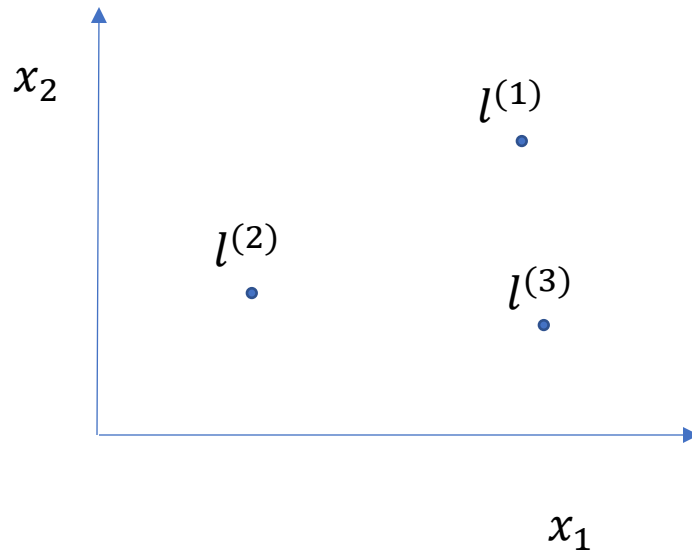


We will define the features $(f_1, f_2, f_3, \dots)$ differently

First step: pick data points on the graph – **landmarks**
 $l^{(1)}, l^{(2)}, l^{(3)}$

Second step: Learn the “best” β ($\beta_0, \beta_1, \beta_2$)

Kernel function



Given x :

$$f_1 = \text{similarity}(x, l^{(1)}) = e^{-\frac{\|x - l^{(1)}\|^2}{2\sigma^2}}$$
$$f_2 = \text{similarity}(x, l^{(2)}) = e^{-\frac{\|x - l^{(2)}\|^2}{2\sigma^2}}$$
$$f_3 = \text{similarity}(x, l^{(3)}) = e^{-\frac{\|x - l^{(3)}\|^2}{2\sigma^2}}$$

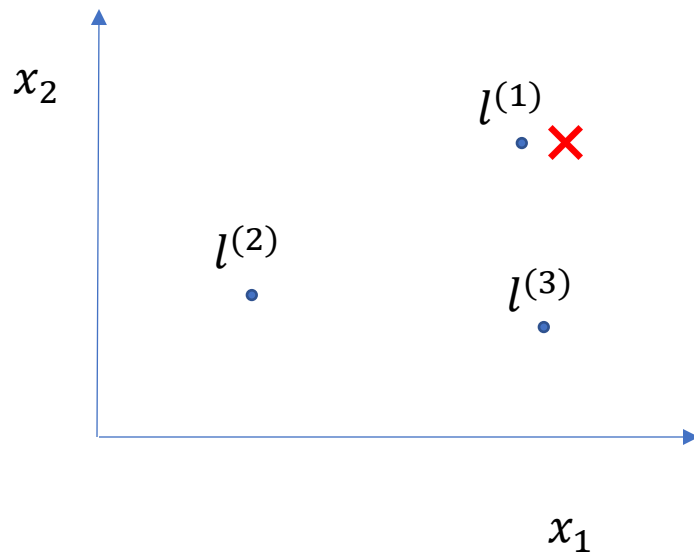
We will define the features $(f_1, f_2, f_3, \dots)$ differently

First step: pick data points on the graph – **landmarks**
 $l^{(1)}, l^{(2)}, l^{(3)}$

Second step: Learn the “best” β ($\beta_0, \beta_1, \beta_2$)

Third step: predict “1” when:
 $\beta_0 + \beta_1 f_1 + \beta_2 f_2 + \beta_3 f_3 \geq 0$

Example



predict "1" when:

$$\beta_0 + \beta_1 f_1 + \beta_2 f_2 + \beta_3 f_3 \geq 0$$

$$\beta_0 = -0.5$$

$$\beta_1 = 1$$

$$\beta_2 = 1$$

$$\beta_3 = 0$$

$$f_1 \approx 1$$

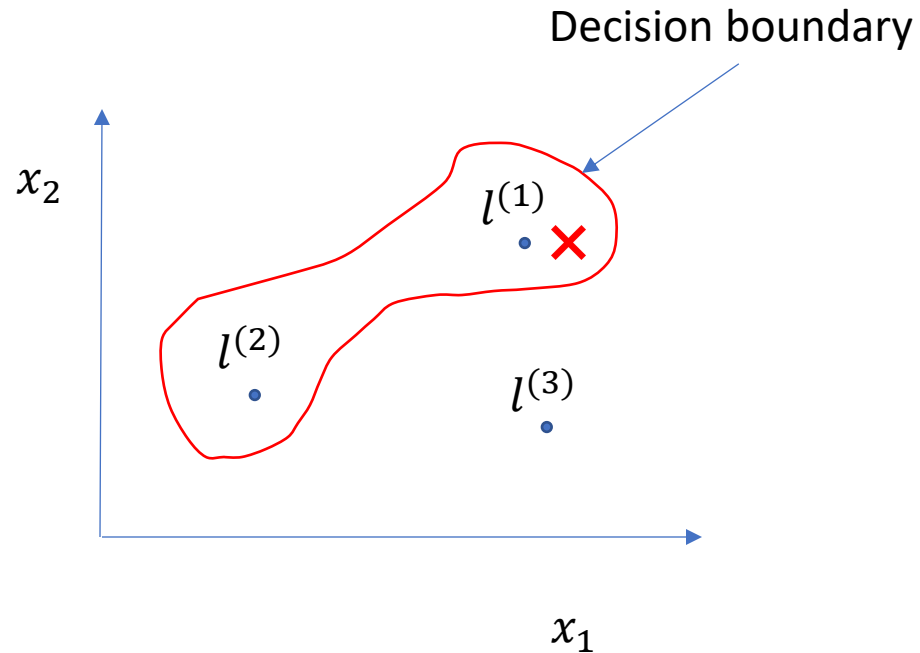
$$f_2 \approx 0$$

$$f_3 \approx 0$$

$$\beta_0 + \beta_1 f_1 + \beta_2 f_2 + \beta_3 f_3$$

$$= -.5 + 1 = 0.5 \geq 0 \rightarrow \text{predict 1}$$

Example



predict "1" when:

$$\beta_0 + \beta_1 f_1 + \beta_2 f_2 + \beta_3 f_3 \geq 0$$

$$\beta_0 = -0.5$$

$$\beta_1 = 1$$

$$\beta_2 = 1$$

$$\beta_3 = 0$$

$$f_1 \approx 1$$

$$f_2 \approx 0$$

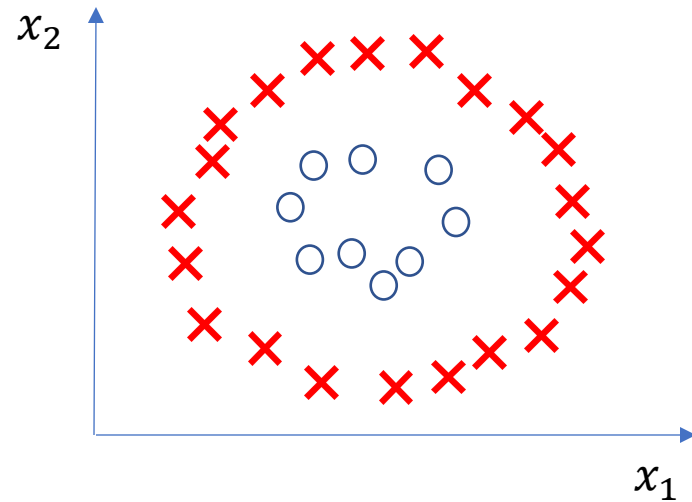
$$f_3 \approx 0$$

$$\beta_0 + \beta_1 f_1 + \beta_2 f_2 + \beta_3 f_3$$

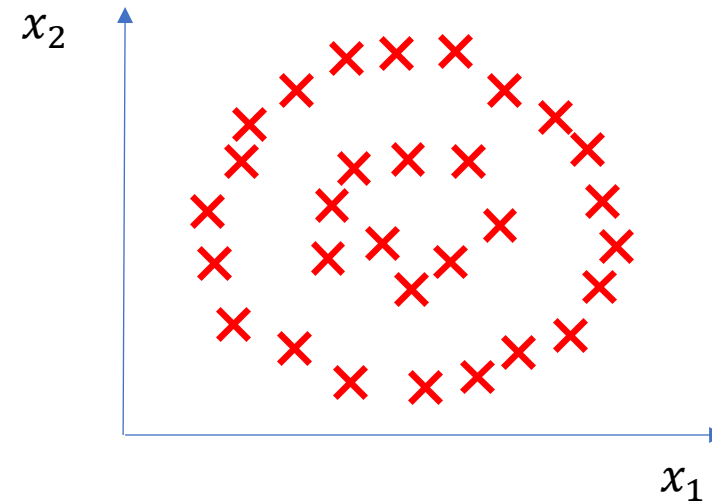
$$= -0.5 + 1 = 0.5 \geq 0 \rightarrow \text{predict 1}$$

How to choose the landmarks

M Training examples



M Landmarks



How to choose the landmarks

We have a minimization problem:

$$\text{Minimize } C \sum_{i=1}^m y^{(i)} \text{cost}_1(\beta f^{(i)}) + (1 - y^{(i)}) \text{cost}_0(\beta f^{(i)}) + \frac{1}{2} \sum_{j=1}^m \beta_j^2$$

$f^{(i)}$ is the following vector: $[k(x_i, l_1), k(x_i, l_2), k(x_i, l_3), \dots, k(x_i, l_m)]$

If C is a big number, my goal will be to make

$\sum_{i=1}^m y^{(i)} \text{cost}_1(\beta f^{(i)}) + (1 - y^{(i)}) \text{cost}_0(\beta f^{(i)})$ as close as can be to 0 $\rightarrow$ have M meaningful landmarks

If C is a small number, my goal will be to make

$\frac{1}{2} \sum_{j=1}^m \beta_j^2$ as close as can be to 0 $\rightarrow$ have less landmarks (lots of β_j will be equal to 0)

Kernel SVM parameters

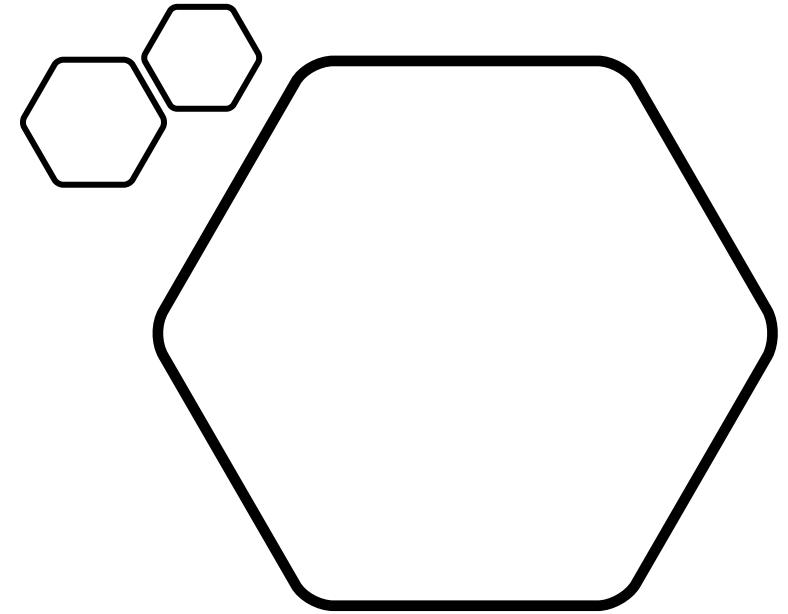
how much you want to penalize the misclassified points

C: Large C – Lower bias (overfitting)

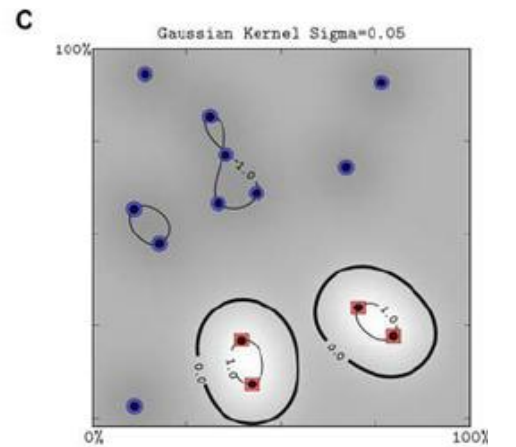
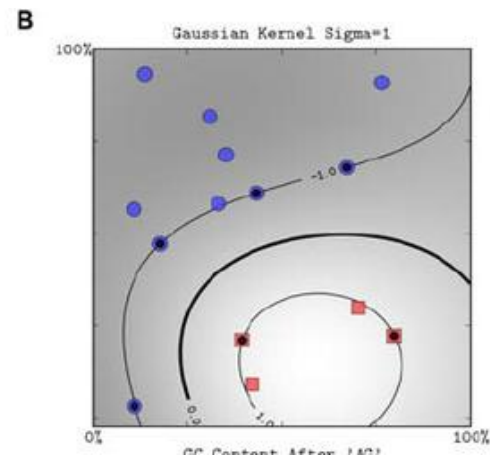
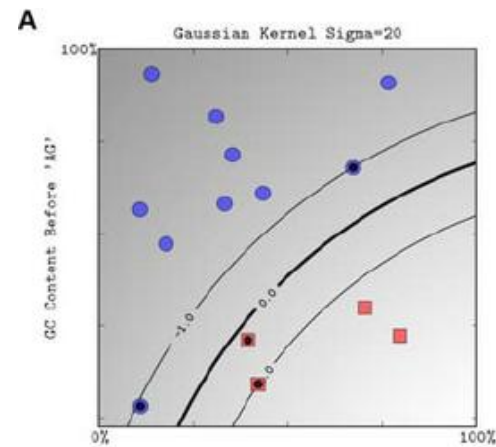
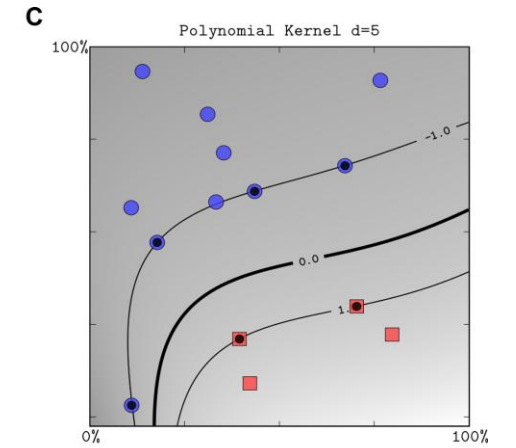
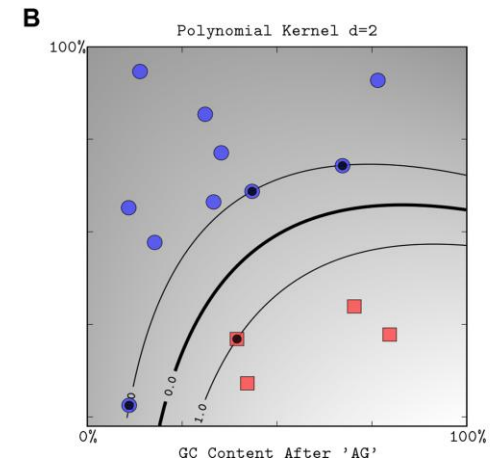
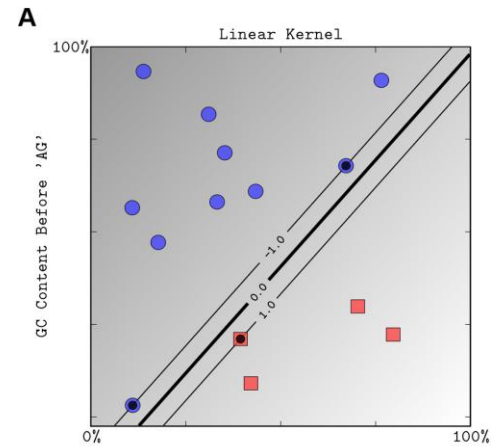
Small C – Higher bias (underfitting)

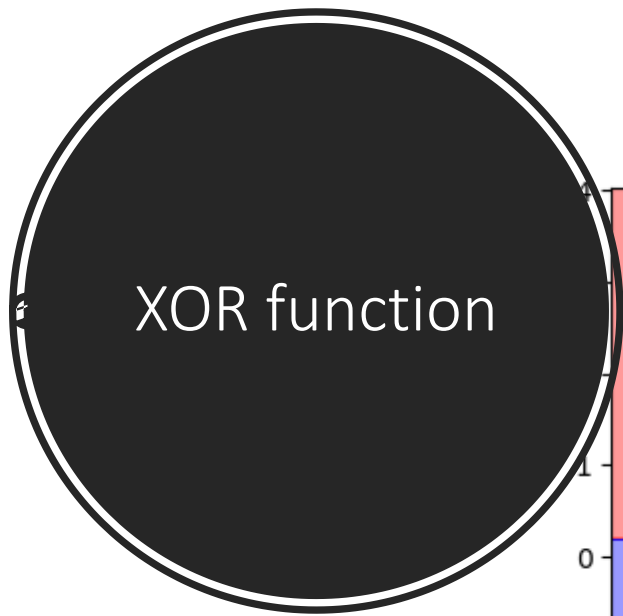
σ^2 : Large σ^2 – Higher bias (underfitting)

Small σ^2 – Lower bias (overfitting)

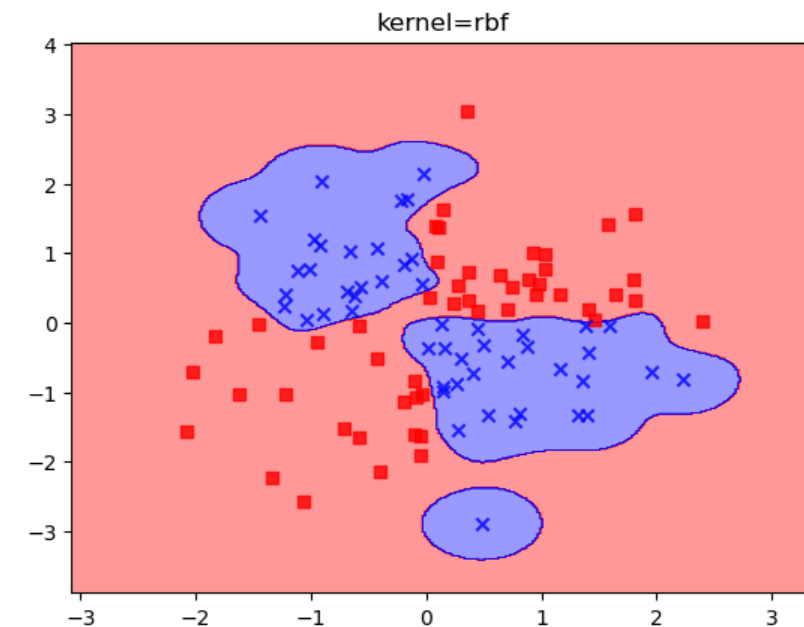
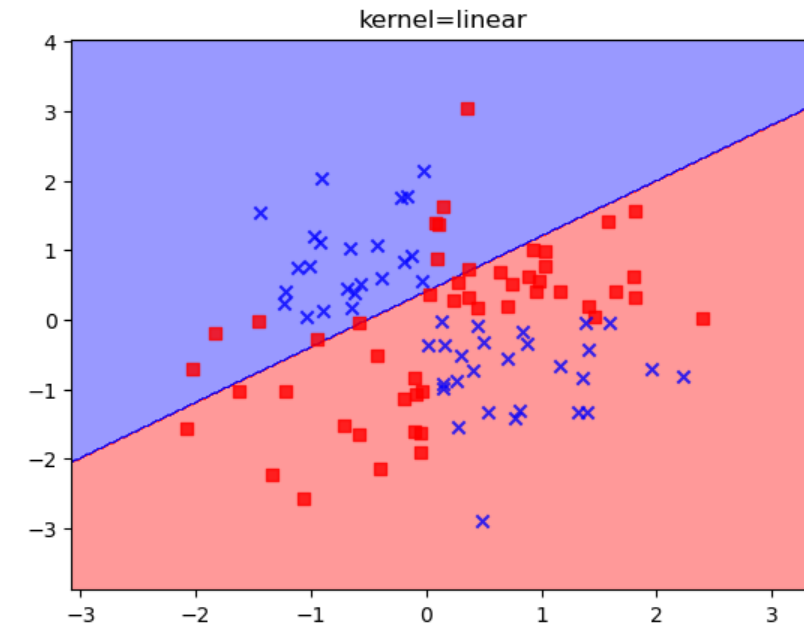
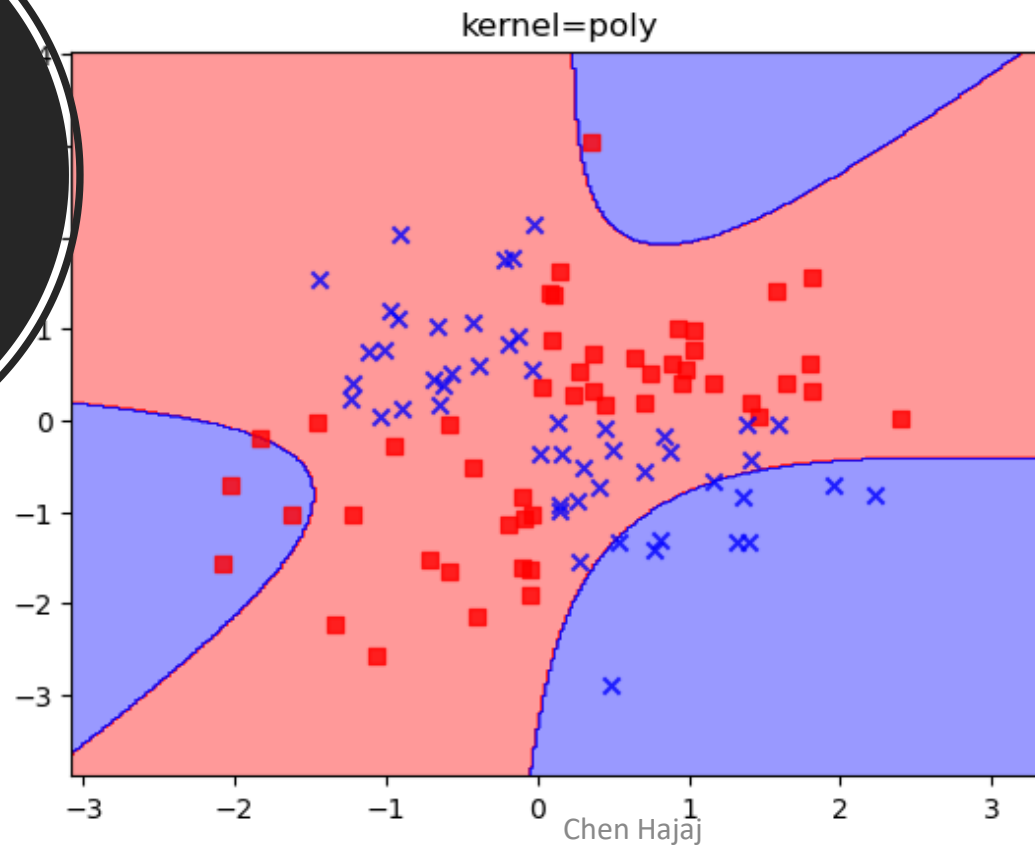


Types of kernel functions





USING EXCLUSIVE OR (XOR) IN CRYPTOGRAPHY			
XOR LOGIC	0 XOR 0 = 0	Same Bits	
	1 XOR 1 = 0	Same Bits	
	1 XOR 0 = 1	Different Bits	
	0 XOR 1 = 1	Different Bits	
XOR Symbol		$\oplus$	



Pros

- **It is effective in high dimensions:** SVM performs well even when the number of features is large. It can handle complex problems with many dimensions, making it suitable for tasks with a high-dimensional feature space.
- **Effective when the number of features is more than training examples:** SVM can handle cases where the number of features is greater than the number of training examples. It avoids the "curse of dimensionality" problem that affects some other algorithms in such scenarios.
- **Best algorithm when classes are separable:** SVM works best when the classes are well separable. It can find the optimal hyperplane that maximizes the margin between the classes, resulting in good classification performance.
- **The hyperplane is affected by only the support vectors:** SVM focuses on the support vectors, which are the data points closest to the decision boundary. This makes SVM more robust to outliers and noise in the data. Outliers have less impact on determining the hyperplane.
- **SVM is suited for binary classification:** SVM is originally designed for binary classification tasks, where the goal is to separate data into two classes. However, it can also be extended to handle multi-class classification problems.

Cons

- **Requires a large amount of time for larger datasets:** SVM can become computationally expensive for larger datasets, especially when the number of data points is substantial. The training time and memory requirements can increase significantly, making it less efficient for large-scale problems.
- **Does not perform well in case of overlapped classes:** When classes overlap significantly, SVM may struggle to find an optimal hyperplane. It relies on maximizing the margin between classes, and if there is substantial overlap, the performance may degrade.
- **Selecting appropriate hyperparameters is challenging:** SVM has several hyperparameters, such as the regularization parameter (C) and the kernel parameters, which need to be carefully tuned. Finding the right values that allow for sufficient generalization performance can be challenging and may require experimentation or domain expertise.
- **Selecting the appropriate kernel function can be tricky:** SVM relies on kernel functions to transform data into a higher-dimensional space. Choosing the right kernel function can be challenging, as different data distributions may require different kernels. It requires domain knowledge and experimentation to select the most suitable kernel.